软件设计实践

软件设计实践笔记

http://pku-software.github.io
mayun@pku.edu.cn
taoxie@pku.edu.cn

Lecture01-从C到C++(1) 数据抽象与类型系统

1-C++类型系统

Definition

数据类型：一组性质相同的值的集合以及定义在这个值集合上的一组操作

C++是静态类型语言

- 在编译时可确定类型
- 类型标识

```
char s[10] = "hello"; // s为char[10]类型
int* P = &b; // p为int *类型
int *p_1, p_2; // p_1为int *类型, p_2为int类型
```

- 类型别名：`using 别名 = 类型标识`

```
using IntPointer = int*;
IntPointer p = &b;
```

1.1-变量声明与初始化

1.1.1-变量声明

Question

变量声明时发生了什么事情?

1. 找到一块能够承载变量类型所需长度的内存空间
2. 将变量名绑定到这块内存空间

3. 对这块内存空间的内容进行初始化

> **Important**
>
> 操作未初始化变量可能造成不确定性行为

1.1.2-初始化方式

```
// T为数据类型，v为值
T a1 {v};
T a2 = {v};
T a3 (v);
T a4 = v;
```

C++11的统一初始化： 所有数据类型均可用列表形式初始化
统一初始化的优点：防止窄缩

```
char c1 = 11111; // 警告
char c2{11111};  // 错误
char c3{100};    // 允许6
```

1.2-引用类型

```
int n = 4;
int& r = n; // r引用了，r的类型是int &
r = 3;
std::cout << r << std::endl; // 3
std::cout << n << std::endl; // 3
n = 5;
std::cout << r << std::endl; // 5
```

> **Caution**
>
> 1. 定义引用时一定要初始化为某个变量的引用
> 2. 初始化后，它就一直引用该变量，不能再引用其他变量

```
double a = 4, b = 5;
double& r1 = a;
double& r2 = r1;
r2 = 10;
cout << a << endl; // 10
```

```
r1 = b;
cout << a << endl; // 5
```

引用和指针的区别：几乎一样，但引用更方便

```
int n = 4;
int& r = n;
int* p = &n;
```

1.2.1-函数传参的值传递与引用传递

不同的交换方式

```
void falseSwap(int a, int b) {
        int tmp;
        tmp = a;
        a = b;
        b = tmp;
} // 失败
void pointerSwap(int* a, int* b) {
        int tmp;
        tmp = *a;
        *a = *b;
        *b = *tmp;
} // 成功
void referenceSwap(int& a, int& b) {
        int tmp;
        tmp = a;
        a = b;
        b = tmp;
} // 成功
```

1.2.2-引用作为函数的返回值

```
int n = 14;
int& SetValue() {
        return n;
} // 返回引用的函数可以作为左值被赋值
int main() {
        SetValue() = 40;
        std::cout << n << std::endl; // 40
        return 0;
}
```

1.3-类型推导

- `auto`：用于定义变量，编译器可以自动判断变量的类型

```
auto i = 100;      // int
auto p = new A(); // A *
auto k = 34343LL; // long long
```

2-常量与只读变量

Definition

常量：在编译期间就能确定的值，且在运行期间这个确定的值不会发生变化，例如：

- 字面量
- 字面量构成的式子

只读变量：用const关键词修饰类型说明符的变量，在运行过程中该变量的值不能改变

- const变量必须初始化
- 与常量的关系
  - 并非所有的只读变量都是常量，只有使用常量作为初始化值初始 化的只读变量才是常量

```
const int a {42};
const int b {a};
const int c {a + b};
// 这些变量均为常量
```

但有的只读变量不是常量

```
int a;
std::cin >> a;
const int b {a};
```

声明常量的方法：

```
constexpr int a {42};
```

const的用法

复杂特殊情况

1. 只读引用

```
int n;
const int& r = n;
r = 200; // error
n = 300; // success
```

`r`为`n`的只读引用，只能读取`n`的值，不能通过`r`修改`n`的值，但是`n`本身可以改变

> ⚠ **Caution**
>
> - T &类型的引用，T类型的变量和常量，const T类型的只读变量，都可以用来初始化const T &类型的引用
> - T类型的常量、const T类型的只读变量、const T & 类型的引用则不能用来初始化T &类型的引用，除非进行强制类型转换

2. const指针
3. 指向只读变量的指针 `const T *`
    - 不可以通过该指针修改其指向的内容
    - 变量不需要只读，但编译器认为这样的指针指向的变量是只读的
    - 不能把`const T *`赋值给`T *`，但反过来可以

```
int n = 3. m = 6;
const int* p = &n;
*p = 5; // 错误，指向只读变量的指针不能修改指针对应的值
n = 4; // 正确，变量本身不是只读的，可以修改
p = &m; // 正确，指向只读变量的指针的指向可以变化
// 即，pointer本身不是只读的，但 *pointer 被视为只读变量
```

```
const int* p1;
int* p2;
p1 = p2; // 正确
p2 = p1; // 错误
p2 = (int *)p1; // 强制类型转换后即可
```

2. 只读的指针变量`T *const`
    - 该指针初始化完成后，就不能更改内容，即不允许指向其他对象

```
int buff_size = 512;
int* const pointer = &buff_size;
// pointer将始终指向buff_size

*pointer = 1024;
// 允许通过这个指针修改
// 但是不能修改pointer指向的变量
```

```
// 即，pointer本身是只读变量，但 *pointer 可修改

const double pi = 3.14;
const double* const pt_pi = &pi;
```

Tip

函数参数为只读指针时，可避免函数内部不小心改变函数指针所指向部分的内容

- 只读引用也有类似的效果

```
void myPrintf(const char* p) {
        strcpy(p, "this"); // error!
        std::cout << p; // no problem!!
}
```

3-动态内存分配

声明并定义数组时，要求数组的大小必须是**常量**，而且要求是**正整数**

- 注意要求是常量而非只读变量
- 当有初始化器时，可省略不写大小

```
int n = 5;
int a[n] {}; // 严格的编译器会报错

const int m = n;
int a[m] {}; // 严格的编译器会报错
```

Question

如何根据需要开出特定大小的数组?

- 动态分配变量
  - `T` 是任意类型名，`p` 是类型为 `T *` 的指针

```
T* p;
p = new T;
```

```
int* pn;
pn = new int;
```

```
*pn = 5;
```

- 动态分配数组
  - `T`是任意类型名，`p`是类型为`T *`的指针，`N`是要分配的个数，也可以是unsigned int型表达式

```
T* p;
p = new T[N];
```

```
int* pn;
int i = 5;
pn = new int[i * 20];
p[0] = 20;
// 编译没问题，运行时会越界（index: 0~99）
p[100] = 30;
```

> ⚠ **Warning**
>
> 用`new`动态分配的内存空间，一定要用`delete`运算符释放

- 如果没有释放，那么即使程序运行结束，这部分内存也不会被回收
  - 即 内存泄露

```
int* p = new int;
*p = 5;
delete p;
delete p; // 异常，一片空间不能被delete多次
```

```
int* p = new int[20];
p[0] = 1;
delete [] p; // 释放数组指针要加[]
```

> **Note**
>
> - **正确性/健壮性**
> - 静态类型可以在编译时发现问题
> - 只读变量能防止错误的数据修改
> - **可扩展性**
> - 动态内存分配可以按需申请内存空间，应对存储数据规模大小的变化
> - **可复用性**
> - 使用变量可以避免程序中重复字面量

- **可理解性**
  - 静态类型可以使开发人员明确知晓每个变量的类型
  - 类型别名可以方便明确变量的类型
  - 自动类型推断能简化开发，但一定程度上破坏了可理解性

Lecture02-从C到C++(1) 过程抽象与函数

函数

函数的声明与定义

- 函数在调用前必须声明
  - 可以先声明后定义
  - 可以多次声明一个名字（但不能多次定义）
- 函数声明时，形参名可以省略

```
void printNum(int x);
// 或者
void printNum(int);

int main() {
        printNum(42);
        return 0;
}

void printNum(int x) {
        std::cout << x << std::endl;
}
```

函数的类型

- 返回值类型(形参类型)

```
void printNum(int x);  // type: void(int)
int max(int x, int y); // type: int(int, int)
```

- 可以用 `using` 起别名

```
using Intof2Int = int(int, int);

int add(int a, int b);

int main() {
```

```
        Intof2Int *funcPtr = &add;
        std::cout << (*funcPtr)(3, 4) << std::endl;

        return 0;
}
```

函数的规约

- 函数实现者和函数使用者之间的约定，包含：
  - 前置条件：对输入参数的规定
  - 后置条件：对函数在不同输入条件下的返回值和其他额外影响的规定
    - return 什么？
    - 修改了什么全局变量？
- 规约的描述方式
  - 函数签名
    - 函数名，参数类型，返回值类型...
  - 函数描述
    - 功能
    - 对参数有什么限制条件
    - 抛出什么异常
    - 其他影响
- 在给定的规约下
  - 函数实现的正确性：对于所有满足前置条件的输入、函数的返回值和额外影响都满足后置条件的约定
  - 函数实现者：根据规约实现
    - 无需关心函数如何调用，可以假设输入始终有效
  - 函数调用者：根据前置条件调用
    - 无需关心函数如何实现，可以假设函数始终正确

函数设计原则

- 每个函数只做一件事情
- 不要写重复代码
- 函数需要具有一定的泛化性

```
double areaSquare(double r) {
        return r * r;
}
double area_circle(double r) {
        return r * r * M_PI;
}
double area_hexagon(double r) {
```

```
        return r * r * (3 * sqrt(3) / 2);
}
```

泛化?

```
double area(double r, double shape_constant) {
        return r * r * shape_constant;
}

double area_square(double r) {
        return area(r, 1);
}

double area_circle(double r) {
        return area(r, M_PI);
}
double area_hexagon(double r) {
        return area(r, (3 * sqrt(3) / 2);
}
```

函数的高级特性

缺省参数

考虑场景

```
void drawCircle(int radius, int x, int y);
```

要画不同颜色的圆

```
void drawCircle(int radius, int x, int y, int color);
```

以前的语句都要加上 `BLACK`，但如果

```
void drawCircle(int radius, int x, int y, int color = BLACK);
```

就可以设置缺省值为 `BLACK`

- C++中，定义函数时可以给最右边的连续若干个参数指定缺省值；调用函数时会默认初始化相应参数
-

函数重载

考虑下面的场景

```
void printInt(int i) {
        std::cout << i << std::endl;
}
void printFloat(float f) {
        std::cout << f << std::endl;
}
void printChar(char c) {
        std::cout << c << std::endl;
}
void printCString(const char *s) {
        std::cout << s << std::endl;
}
```

函数重载

同一作用域中名字相同但形参列表不同的多个函数

- 形参列表不同包括参数个数不同或（和）参数类型不同
- 编译器会根据调用语句中的实参个数和类型判断应该调用哪个函数
  - 但可能会由于类型自动转换或者默认参数导致有多个函数可以匹配而产生二义性问题，ERROR
- 同名函数只有参数列表不同才能算重载，如果仅仅是返回值类型不同则不构成重载

复杂特殊情况

const的用法

- `T` 和 `const T` 会被认为是相同的类型
- `T *` 和 `T * const` 会被认为是相同的类型
- `T &` 和 `const T &` 会被认为是不同的类型
- `T *` 和 `const T *` 会被认为是不同的类型

函数指针

求和 $\sum_{k=1}^{n} k$，$\sum_{k=1}^{n} k^3$，$\sum_{k=1}^{n}(4k-3)(2k+5)$
合并为

$$\mathcal{S}[n, f] = \sum_{k=1}^{n} f(k)$$

Definition

指向函数的指针变量称为“函数指针”

声明形式

```
int (*ptr)(int, int);
```

- 指针名的括号不能省略
- 声明ptr是一个函数指针，及其参数和返回值类型
-

使用方法

```
int max(int, int);
int (*ptr)(int, int);
ptr = &max;
(*ptr)(3, 5);
```

利用类型别名简化

```
int max(int, int);
using funcType = int(int, int);
funcType* ptr = &max;
(*ptr)(3, 5);
// 也可以（语法糖）
funcType* ptr;
ptr = max;
ptr(3, 5);
```

函数指针作为形参

```
int summation(int n, int (*term)(int)) { // 高阶函数
        int total = 0, k = 1;
        while(k <= n) {
                total += term(k);
                k++;
        }
        return total;
}
```

Lambda表达式

- 只调用一次的简单函数，可以用lambda表达式
    - 匿名函数，可以理解为函数类型的字面量
- 声明方式
    - [捕获列表](参数表) -> 返回值类型 函数体

    - 函数体中可以访问全局变量，但不能访问局部变量，如果想要访问局部变量，需要用捕获列表（使用捕获列表后无法转换为函数指针）
        - [ ] 不使用任何外部变量
        - [=] 外部变量传值
        - [&] 外部变量传引用
        - [x, &y] x传值，y传引用
        - [=, &x, &y] x，y传引用，其余传值
        - [=, x, y] x，y传值，其余传引用
    - 返回值类型可以不写
- 捕获列表示例

```
int main() {
        int x = 100, y = 200, z = 300;
        cout << [](double a, double b) { return a + b; }(1.2, 2.5) << endl;
        auto ff = [=, &y, &z](int n) {
                cout << x << endl;
                y++;
                z++;
                return n * n;
        }
        cout << ff(15) << endl;
        cout << y << "," << z << endl;
}
```

> ⚠ **Warning**
>
> 没有捕获列表的lambda表达式才能作为函数指针传入高阶函数

> 🖊 **Note**
>
> - **正确性/鲁棒性**
> - 函数规约使得函数实现者和调用者之间消除歧义，减少错误
> - 函数重载存在二义性问题，但静态检查可消除此问题
> - **可扩展性**
> - 函数规约使得函数实现者和调用者只要遵循规约就可以独立的变化和替代
> - 缺省参数和函数重载可使得增加函数的参数时不会影响已有的函数调用代码》
> - **可复用性**
> - 函数使得相同计算逻辑的程序代码无需多次书写（DRY原则）
> - 函数重载使得函数名称可被复用
> - 利用函数指针实现的高阶函数可使得处理逻辑得到更高级别的复用
> - **可理解性**

- 良好的函数命名有助于准确理解其功能
- 查看规约比审查实现更容易理解程序的功能
- 函数重载可能会降低一定的可理解性，要谨慎考量是否需要重载
- Lambda表达式虽可以减少代码书写，但一定程度降低了可理解性

补充：Lambda

不同初始化器效果有差异

格式建议left-alignment

```
int* a;
int& b = c;
```

如何声明持有函数指针的数组

`ReturnType (*arrayName[N])(ParamType);`
可以使用`using`

```
using FuncType = double(double, double);
FuncType* funcArray[10];
```

悬垂引用

```
int& f(int x) {
        return x;
} // 返回了不存在的变量的引用
```

命名空间

- `using namespace std;`但是不好
- `using std::cin, std::cout;`

`new`和`delete`运算符

- `new`和`new[]`运算符：开辟一块内存空间，返回一个指向这片空间的指针
- `int* a = new int;`
- `int* p = new int[5];`
- 使用`new`申请的内存空间需要用`delete`或`delete[]`运算符释放，否则会造成内存泄漏
- `delete a;`
- `delete[] p;`

- 现代操作系统会在运行结束后自动释放

Lambda表达式

- Lambda表达式的类型没有名字，无法写出类型
  - 每一个Lambda表达式有唯一的名字
- 只能用auto来声明
- 不带捕获的Lambda表达式可以隐式转换成函数指针

```
auto f = [](int x) { return x * x; };
using FuncType = int(int);
FuncType* funcPtr {f};
```

```
auto add = [](int x) {
        return [=](int y) {
                return x + y;
        };
};
std::cout << add(42)(16) << std::endl;
```

Lecture03-C++面向对象程序设计（1）类与对象初步

复杂的数据抽象

- 可以利用结构体

```
struct Rational {
        int numer; // 分子
        int denom; // 分母
};
```

> **Question**
>
> 结构体实现抽象有什么问题？

- 对使用者不友好

类/Class

- 将数据结构和操作该数据结构的函数“捆绑”在一起
- 类的成员：

    - 成员变量
    - 成员函数

```
class Rational {
        int num;
        int den;

        void init(int n, int d) {
                num = n; den = d;
        }
        int numer() { return num; }
        int denom() ( return den; )
};
```

类的定义与使用

- 访问范围说明符：(无顺序要求，可以出现多次)
    - `public`
    - `private`(默认)
    - `protected`
- 成员变量的定义和普通变量相同
- 成员函数的定义和普通函数声明相同
    - 一个类的成员函数之间可以互相调用
    - 成员函数中可以访问成员变量
    - 成员函数可以重载，也可以设定默认值
    - 成员函数的实现可以位于类定义之外
- 成员函数和成员变量出现的先后次序没有规定
  示例：
- 在类的定义外实现成员函数，函数名为`ClassName::FuncName`

```
class CRectangle {
public:
        int w, h;
        int area();
        int perimeter();
        void init(int w_, int h_);
};

int CRectangle::area() { return w * h; }
int CRectangle::perimeter() { return 2 * (w + h); }
void CRectangle::init(int w_, int h_) {
        w = w_;
        h = h_;
}
```

对象的定义

- 通过类定义 `ClassName ObjName;`
- 每个对象有各自的存储空间保存其成员变量
  - 不会互相影响
- 成员函数在内存中只有一份，但是可以作用于不同的对象

> **Question**
>
> 如何访问对象的成员？

- `ObjName.Attribute` 或 `ObjName.Method()`
- `ObjPtr->Attribute` 或`ObjPtr->Method()
- `ObjRef.Attribute` 或 `ObjRef.Method()`

```
CRectangle r1;
CRectangle* ptr_r1 = &r1;
CRectangle& ref_r1 = r1;
```

> **Question**
>
> 成员的访问范围？(被访问)

- 不同关键字：
  - `private`：私有成员，只能在同一个类的成员函数内被访问
  - `public`：公有成员，可以在任何地方被访问
  - `protected`：
  - 没有关键字被认为是 `private`
- 在类的成员函数内部，能够访问：
  - 当前对象的全部属性，函数
  - 同类其他对象的全部属性，函数
- 在类的成员函数以外的地方，只能够访问该类对象的公有成员
- 设置私有成员可以实现信息隐藏/Information Hidden，作用：
  - 强制：对成员变量的访问只能通过成员函数
  - 方便修改成员变量的类型等属性，只需要成员函数
- `struct` 和 `class` 的区别
  - 使用 `struct` 定义的类，默认是公有

对象的生命周期

回顾：变量的生命周期

- 全局变量：未明确初始化时会被自动初始化为全0，程序结束前被销毁
- 局部变量：未明确初始化时初始值随机，在函数执行结束后被销毁
- 函数传参：执行复制初始化，在函数执行结束后被销毁
- 函数返回值：自动生成临时变量(若开启返回值优化RVO则不生成)，执行复制初始化，返回完成后立刻被销毁

构造函数

Question

定义初始化函数 `ObjName.init(arguments, ...)` 有什么问题?

- 规约复杂
- 需要显式调用

> **Definition**
> 构造函数是一种特殊的成员函数，用于初始化对象

- 函数名字与 `ClassName` 完全相同，可以有参数，但不能有返回值(`void` 也不行！！！)
- 如果没写构造函数，编译器会生成一个默认无参数的构造函数
  - 默认构造函数无参数，不做任何操作
  - 如果定义了构造函数，则编译器不生成默认的无参数的构造函数
- 对象生成时构造函数自动被调用。对象一旦生成，就再也不能在其上执行构造函数
- 构造函数可以重载

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i = 0);
};
Complex::Complex(double r, double i = 0) {
        real = r;
        imag = i;
}

Complex c1;                         // Error, no parameters
Complex* ptr_c = new Complex;       // Error, no parameters
Complex c2(2);                      // OK
Complex c3(2, 4), c4(3, 5);         // OK
Complex* ptr_c = new Complex(3, 4)  // OK
```

复制构造函数

- 如果一个构造函数只有一个参数，这个参数是对同类对象的引用，这样的构造函数称为复制构造函数
  - 形如 `ClassName::Constructor(ClassName& ObjName)` 或 `ClassName::Constructor(const ClassName& ObjName)`，后者能以只读对象作为参数
- 如果没有定义复制构造函数， 编译器会生成默认复制构造函数
  - 默认的复制构造函数依次对每个成员复制初始化
    - 如果是基础数据类型，则直接复制过去；如果是结构体类型，则调用它的复制构造函数
  - 如果定义了自己的复制构造函数，则不会生成默认的复制构造函数
- 不允许有形如 `ClassName::Constructor(ClassName)` 的构造函数(二义性，套娃)

```
class Complex {
private:
        double real, image;
};
Complex c1;     // 调用默认的构造函数
Complex c2(c1); // 调用默认的复制构造函数，让c2初始化为和c1一样的对象
```

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i = 0) {
                real = r;
                imag = i;
        }
        Complex(const Complex& c) {
                real = c.readl;
                imag = c.imag;
                std::cout << "Copy Constructor called" << std::endl;
        }
};
Complex c1(3, 5);
Complex c2(c1) // Copy Constructor called
```

- 复制构造函数起作用的三种情况

1. 用一个对象初始化同类的另一个对象

```
Complex c2(c1);
Complex c2 = c1;
```

2. 如果函数有一个参数是类的对象，该函数被调用时，类的复制构造函数将被调用
    - 大开销！

```
void func(Complex c) {}  // Copy Constructor called!
void func(Complex& c) {} // OK!
```

3. 函数的返回值是对象，则函数返回时，类的复制构造函数被调用 ```

```
ClassName Func() {
        ClassName Obj(parameters, ...);
        return Obj;
};
int main() {
        std::cout << Func().value << std::endl;
        return 0;
}
```

类型转换构造函数

- 显式类型转换：在程序中利用类型转换运算符明确表示类型转换，如
    - 类型(表达式) 或(类型)表达式
- 隐式类型转换：编译器发现某个类型不适合当前语境，从而自动进行类型转换，如
    - 用A类型初始化B类型的变量
        - 声明并定义B类型变量，其中初始化值是A类型的
        - 函数形参期望是B 类型的，而实参是A类型的
        - 函数返回值类型期望是B，而return语句中的表达式是A类型的
    - 某个运算符期待B 类型操作数，但实际给出的操作数却是A类型的
    - 在 `if`， `while` 和 `for` 的条件表达式中，可以将A类型转换到 `bool`
- 构造函数可以用于实现类型转换
    - 被转换的类型与构造函数的参数类型匹配时会自动调用
    - 编译器会自动调用构造函数，建立一个无名的临时对象
- 使用 `explicit` 修饰构造函数，可使该构造函数只能用于显式类型转换，不能用于隐式类型转换
    - 可以用于隐式类型转换的构造函数（即没有用 `explicit` 修饰的构造函数）又被称为类型转换构造函数

```
class Complex {
public:
        double real, imag;
        explicit Complex(int i) {
                std::cout << "IntConstructorCalled" << std::endl;
                real = i; imag = 0;
        }
```

```
        Complex(double r, double i) { real = r; imag = i; }
};

int main() {
        Complex c1(7, 8);
        Complex c2 = Complex(12); // explicit
        c1 = 9 // 无隐式转换构造函数
        c1 = Complex(9) // ok, explicit
        std::cout << c1.real << "," << c1.imag << std::endl;
        return 0;
}
```

析构函数

- 名字与类名相同，在前面加`~`
  - `~ClassName() {}`
  - 没有参数和返回值
  - 一个类最多一个
- 在对象消亡时自动被调用
  - 可以“善后”，delete动态分配的空间...
- 如果定义类是没写析构函数，则编译器自动生成缺省析构函数
  - 缺省析构函数什么都不做
  - 如果定义了析构函数，则编译器不生成缺省析构函数
- 返回值返回后会被调用
  - 返回后临时对象会消亡

```
class MyClass {
public:
        ~MyClass() {
                std::cout << "destructor" << std::endl;
        }
};
MyClass obj;
MyClass Func(MyClass sobj) { // 参数消亡也会调用析构函数
        return sobj;
}
int main() {
        obj = func(obj);
        return 0;
}
```

Note

- 正确性/鲁棒性

        - 成员的访问范围可以实现信息隐藏，只将最少的信息暴露出去，从而可避免非预期的访问
        - 利用构造函数可以实现对象的强制初始化，避免非预期初始值
        - 利用析构函数可以实现对象销毁前的处理，避免内存泄漏等问题
    - 可扩展性
        - 数据使用和实现的分离使得在不影响数据使用的前提下改变数据的实现方式
        - 对象将复杂系统分成独立地部分，不同部分可单独改变而不相互影响
        - 信息隐藏使得对象的变更可以尽量少地影响程序的其他部分
    - 可复用性
        - 类实现了对相同的属性和操作的封装，通过类的实例化得到对象，实现复用
    - 可理解性
        - 对象更加贴近现实世界的情况，使得复杂系统的结构更加清晰
        - 利用封装可以减少全局名字的使用

Lecture04-C++面向对象程序设计(2) 类与对象进阶

对象数组

- 使用 `classType` 类型定义数组

```
class CSample {
        int x;
public:
        CSample() {
                std::cout << "Constructor 1 Called" << std::endl;
        }
        CSample(int n) {
                x = n;
                std::cout << "Constructor 2 Called" << std::endl;
        }
};
int main() {
        CSample array1[2];
        std::cout << "step1" << std::endl;
        CSample array2[2] {4, 5};
        std::cout << "step2" << std::endl;
        CSample array3[2] {3};
        std::cout << "step3" << std::endl;
        CSample* array4 = new CSample[2];
        delete[] array4;
        return 0;
}
```

- 对象数组的析构
  - 对象数组生命期结束时候，每个对象的析构函数会被依次调用
  - `delete` / `delete[]` 也会导致析构函数的调用

this指针

- 成员函数中可以使用的一个特殊指针变量，指向调用成员函数的那个对象

```
class Complex {
        double real, imag;
public:
        void print() {
                std::cout << real << "," << imag << std::endl;
        }
        Complex(double real, double imag) {
                this->real = real;
                this->imag = imag;
        }
        Complex& addOne() {
                this->real++;  // equivalent to "real++"
                this->print(); // equivalent to "print()"
                return *this;
        }
};
int main() {
        Complex c1(1, 1), c2(0, 0);
        c2 = c1.addOne();
        c1.addOne().addOne();
        return 0;
}
```

对象作为成员变量

- 一个类的成员变量是另一个类的对象

```
class CTyre {
        int radius;
        int width;
public:
        CTyre(int r, int w) {
                radius = r;
                width = w;
        }
};
class CEngine {
        // define this class
};
```

```cpp
class CCar {
        int price;
        CTyre tyre; // 初始化在这里，调用了默认构造函数
        CEngine engine;
public:
        CCar(int p, int tr, int tw) {
                price = p;
                tyre = CTyre(tr, tw) // error，这里是赋值
        }
};
```

Important

C++中，成员变量的初始化在调用构造函数之前

- 可以在成员变量声明时附带默认初始化器

```cpp
class CCar {
        int price {0};
        CTyre tyre {0, 0};
        CEngine engine;
public:
        CCar(int p, int tr, int tw) {
                price = p;
                tyre = CTyre(tr, tw) // 开销好大！
        }
};
```

Question

能不能减小开销？

成员初始化列表

- 构造函数提供了一种特殊的语法，在声明后附加一个成员初始化列表

```cpp
ConstructorFunc(parameters): attribute_1 initializer_1, ... {
        ...
}
```

- 如果在构造对象时匹配上了目前这个构造函数，则先根据初始化列表执行成员的初始化，再执行构造函数

- 初始化器可以是大括号的统一初始化，对于成员对象则可使用小括号调用对应的构造函数进行初始化
- 初始化器需要的值可以从构造函数的参数中获取，并且允许参数名和成员变量名具有相同的名字

所以，上边代码可以改为

```
class CCar {
        int price;
        CTyre tyre;
        CEngine engine;
public:
        CCar(int price, int tr, int tw): price{price}, tyre(tr, tw) { }
};
```

> **Note**
>
> 对象初始化一般按照以下的流程：
>
> - 匹配构造函数
> - 如果某个成员在这个构造函数的初始化列表中提及，则按初始化列表中的初始化值初始化这个成员
> - 如果某个成员提供了默认初始化器，则按默认初始化器的初始化值初始化这个成员
> - 如果某个成员不满足上述两条件，则默认初始化它
> - 简单类型什么都不做，类类型调用没有参数的构造函数
> - 当成员全部初始化完毕后，再执行构造函数

对象初始化与销毁的一般流程

- 初始化每个成员的顺序是按照成员列表的顺序执行的（即定义在最“上面”的最先被构造），与初始化列表的书写顺序无关
- 对象销毁时的析构顺序和其构造顺序是相反的
  - 先调用析构函数，再逆序析构每个成员（即写在最“下面”的最先被析构）

```
class CTyre {
public:
    CTyre() { cout << "CTyre contructor" << endl; }
    ~CTyre() { cout << "CTyre destructor" << endl; }
};

class CEngine {
public:
    CEngine() { cout << "CEngine contructor" << endl; }
    ~CEngine() { cout << "CEngine destructor" << endl; }
};

class CCar {
private:
    CEngine engine;
    CTyre tyre;
public:
    CCar() { cout << "CCar contructor" << endl; }
    ~CCar() { cout << "CCar destructor" << endl; }
};

int main() {
    CCar car;
    return 0;
}
```

输出结果

```
CEngine contructor
CTyre contructor
CCar contructor
CCar destructor
CTyre destructor
CEngine destructor
```

- 对于含有成员对象的类，其对象如果是用默认复制构造函数初始化的，那么它里面的成员对象也会用复制构造函数初始化
  - 如果自己写复制构造函数，也需要注意成员对象的初始化问题

```
class A {
public:
    A() { cout << "default" << endl; }
    A(const A &a) { cout << "copy" << endl; }
};

class B {
    A a;
    int p;
};

int main() {
    B b1, b2(b1);
    return 0;
}
```

编译器给B生成的默认复制构造函数类似：

```
B(const B &t): a(t.a), p{t.p} {}
```

输出结果

```
default
copy
```

静态成员

- 在定义前面加了static关键字的成员
- 作用：类似全局变量或全局函数，将类当做命名空间来用
- 普通成员变量每个对象有各自的一份
  - 静态成员变量对于每个类就只有一份
- 普通成员函数必须具体作用于某个对象

    - 静态成员函数并不具体作用于某个对象

```
class CRectangle {
private:
        int w, h;
        static int nTotalArea;
        static int nTotalNumber;
public:
        CRectangle(int w_, int h_);
        ~CRectangle();
        static void printTotal()
};
```

- 静态成员不需要通过对象就能访问
    - 除了和普通成员一样采用 `ObjName.Name`， `PointerName->Name`， `RefName.Name` 访问以外
    - 还可以通过类名::静态成员名的方式访问
- 设置静态成员的目的
    - 将与某些类紧密相关的全局变量和函数写到类里面，看上去像一个整体，易于维护和理解

静态成员变量

- 成员列表中的静态成员变量大多只能是声明而非定义
    - 一般情况下，必须在类定义的外面专门对静态成员变量进行声明，同时可以初始化
    - 否则编译能通过，链接不能通过

```
class A {
public:
        static int a;    // 只能声明!
        static int a{0}; // error!
};

int A::a{0}; // 要写int，但不要写static

int main() {
        int A::a = 2;
}
```

静态成员函数

- 在静态成员函数中，不能访问非静态成员变量，也不能调用非静态成员函数
- 静态成员函数中不能使用this指针

```
class A {
        static void f() {
                std::cout << "Hello" << std::endl;
        } // 可以在class里定义
        static void g(); // 也可以在外边定义
};

void A::g() { } // class外定义需要写为 ReturnType ClassName::FuncName()

int main() {
        A::f(); // 如同有namespace
}
```

只读成员

只读对象

- 如果不希望某个对象的值被改变，则定义该对象的时候可以在前面加const关键字，声明为只读对象
    - 只读对象只能使用构造函数、析构函数、有const说明的函数（只读成员函数）
    - 可读取成员变量的值，但不可修改（除非用mutable修饰）
    - 只读对象或只读对象引用作为函数形参，可避免在函数修改对象

```
class Sample {
public:
        int value{5};
        void getValue() { }
};
int main() {
        const Sample obj; // 声明了一个只读变量
        std::cout << obj.value << std::endl; // ok
        obj.value = 10; // error，只读变量的成员函数的值不能修改
        obj.getValue(); // error，不能调用非只读成员函数
}
```

只读成员函数

- 在类的成员函数说明后面可以加const关键字，则该成员函数成为只读成员函数
    - 只读成员函数内部不能改变该类的成员变量的值，也不能调用该类中的非只读成员函数
    - 可需修改静态成员变量的值，也可调用静态成员函数
    - 声明和定义只读成员函数时都必须使用const关键字

```
class Sample {
public:
```

```
        int value;
        void setValue const;
        void func() { };
};
void Sample::setValue() const {
        value = 0; // error，只读成员函数不能修改成员变量的值
        func();    // error，只读成员函数不能调用非只读成员函数
}
int main() {
        const Sample o;
        o.setValue(); // ok，只读对象可以调用只读成员函数
        return 0;
}
```

- 两个函数，名字和参数表都一样，但是一个是const， 另一个不是，这种情况也算作重载

```
class CTest {
        int n;
public:
        CTest() { n = 1; }
        int getValue() const { return n; }
        int getValue() { return 2 * n; }
};
int main() {
        const CTest obj1;
        CTest obj2;
        std::cout << obj1.getValue() << "," << obj2.getValue() << std::endl;
        // 1,2
        return 0;
}
```

mutable成员变量

- 用 `mutable` 关键字修饰的成员变量可以在 `const` 成员函数中修改

```
class CTest {
private:
        mutable int n;
        bool flag;
public:
        bool getData() const {
                n++; // n被mutable修饰，可以被const函数修改
                return flag;
        }
};
```

只读成员变量

- 使用const修饰的成员变量为只读成员变量
- 没有办法通过任何方式被赋值（在构造函数和只读函数中也都不行）
- 必须被显式地初始化，即必须拥有一个默认成员初始化器，或者出现在成员初始化列表中

```
class A {
        const int data{0};
        A () {
                data = 1; // error!
        }
        void f() const {
                data = 1; // error!
        }
};
```

```
class A {
        const int data;
        A(): data{2} { } // 如果声明时没有初始化，就必须使用初始化列表初始化
}
```

友元

- 友元：让某个类以外的函数可以访问这个类的私成员
  - 友元函数：一个类的友元函数可以访问该类的私有成员
  - 友元类：如果A是B的友元类, 那么A的所有成员函数都可以访问B的私有成员
- 声明友元：friend关键字
  - 如果想允许函数f访问类A的私有成员，只需要把f的声明抄一遍放在A的成员列表中，然后加上friend关键字 修饰
  - 声明友元可在类定义的任何位置，不受访问范围关键词的约束

友元函数

```
class CCar;

class CDriver {
public:
        void modifyCar(CCar* pCar);
};

class CCar {
private:
        int price;
        // ”找朋友“（声明该类的友元函数）
        friend int mostExpensiveCar(CCar cars[], int total);
        // 声明函数mostExpensiveCar是CCar类的友元
```

```
        friend void CDriver::modifyCar(CCar* pCar);
        // 声明CDriver类中的modifyCar函数为CCar类的友元
};

void CDriver::modiftyCar(CCar* pCar) {
        pCar->price += 1000;
}

int mostExpensiveCar(CCar cars[], int total) {
        int tmpMax = -1;
        for (int i = 0; i < total; ++i) {
                if (cars[i].price > tmpMax) {
                        tmpMax = cars[i].price;
                }
        }
        return tmpMax;
}
```

友元类

```
class CCar {
private:
        int price;
        friend class CDriver; // 声明CDriver类为友元类
};

class CDriver {
public:
        CCar myCar;
        void modifyCar() {
                myCar.price += 1000;
        } // 由于CDriver类为CCar类的友元类，故可以访问私有成员
};
```

Note

- 正确性/鲁棒性
  - 成员对象需要使用初始化列表进行初始化
  - 只读对象可以防止对象的值被修改
- 可扩展性
  - this指针可以用于返回调用函数的当前对象，可以支持表达式的扩展
  - 友元使得私有成员具备例外访问的能力
- 可复用性
  - 类的成员变量可以是对象，从而复用其他类的属性和操作
- 可理解性

- 使用成员对象可以将对象之间的关系显式化
- 静态成员将与某个类关系密切的全局函数和变量与类关联在一起

Lecture05-C++面向对象程序设计(3)运算符重载

基本概念

- C++预定义了一组运算符，用来表示对数据的运算
  - `+, -, *, /, %, ^, &, ~, !, |, =, <<, >>, !=, ...`
  - 这些运算符只能用于基本数据类型
- 运算符重载：对C++中与定义的运算符赋予其他的用法
  - 扩展运算符使用范围
- 运算符重载的实质：函数重载
  - 重载的函数名是`operator@`，其中@是要重载的运算符
  - 重载的函数可以是成员函数，也可以是非成员函数

```
ReturnType operator@(parameters) {
        ...
}
```

- 运算符的表达式->运算符函数的调用
- 运算符的操作数->运算符函数的参数
- 运算符被多次重载时，根据实参的类型决定调用哪个运算符函数
- 运算符重载的限制
  - 不能创建新的运算符
  - 不能改变运算符的优先级和结合性
  - 不能重载的运算符：`sizeof, ., .*, ::, ?:, typeid`
  - 不能重载的运算符：`&&, ||, ,, ->*, new, delete`
  - 重载的运算符的优先级和结合性与原来的一样
- 运算符重载的优点
  - 使得类的对象可以像基本数据类型一样使用运算符
  - 使得类的对象可以参与运算
  - 使得类的对象可以像基本数据类型一样进行类型转换
  - 使得类的对象可以像基本数据类型一样进行输入输出
- 运算符重载的形式
  - 重载为成员函数
  - 重载为非成员函数

运算符重载的两种形式

1. 形式一：重载为成员函数

- 参数个数为运算符目数 - 1

```
class Complex {
public:
        double real, imag;
        Complex(double r, double i): real{r}, imag{i} {}

        Complex operator+(const Complex& c) {
                return Complex(real + c.real, imag + c.imag);
        }
        Complex operator-(const Complex& c) {
                return Complex(real - c.real, imag - c.imag)
        }
};
```

2. 形式二：重载为普通函数

- 参数个数为运算符目数

```
class Complex {
public:
        double real, imag;
        Complex(double r, double i): real{r}, imag{i} {}
};
Complex operator+(const Complex& c1, const Complex& c2) {
        return Complex(c1.real + c2.real, c1.imag + c2.imag);
}
Complex operator-(const Complex& c1, const Complex& c2) {
        return Complex(c1.real - c2.real, c1.imag - c2.imag)
}
```

赋值运算符的重载

- 赋值运算符“=”要求左右两个操作数的类型是匹配的，或者可通过类型转换实现类型匹配
- 有时候希望“=”两边操作数的类型即使不兼容也能够成立，就需要对“=”进行重载
- C++规定，赋值运算符“=”只能重载为成员函数

例：编写一个长度可变的字符串类String，包含一个 `char*` 类型的成员变量只想动态分配的存储空间，存储C风格字符串

```
class String {
private:
        char* str;
public:
```

```
        String(): str{new char[1]} { str[0] = '\0'; }
        const char* c_str() const { return str; }
        String& operator=(const char* s);
        ~String() { delete[] str; }
};
String& String::operator=(const char* s) {
        delete[] str;
        // 必须delete再重新分配
        // 避免内存泄漏
        str = new char[strlen(s) + 1];
        strcpy(str, s);
        return *this;
}
```

- 同时包含类型转换构造函数和相同类型的赋值运算符重载时，优先使用赋值运算符重载

```
class String {
        String(const char* s) {
                str = new char[strlen(s) + 1];
                strcpy(str, s);
        }
};
```

预置赋值运算符重载

当没有在类里定义赋值运算符重载时，编译器会生成一个预置的赋值运算符重载，使得同一个类的对象可以使用=赋值

- 预置赋值运算符重载类似：

```
String& String::operator=(const String& s) {
        str = s.str;
        return *this;
}
```

Question

这样有什么问题？

- 浅拷贝

上述预置赋值运算符重载导致浅拷贝，即两个对象的str指向同一块内存，当一个对象的str被释放时，另一个对象的str也会变成野指针；一个对象的str被修改时，另一个对象的str也会被修改

可以自己定义赋值运算符重载，使得两个对象的str指向不同的内存

```
String& String::operator=(const String& s) {
        delete[] str;
        str = new char[strlen(s.str) + 1];
        strcpy(str, s.str);
        return *this;
}
```

Question

这样有什么问题?

如果考虑如下语句

```
String s = "Hello";
s = s;
```

- 会发生内存泄漏
- 会发生str指向自己的情况(套娃)

加上判断

```
if (this == &s) { return *this; }
```

返回值使用什么?

- 为什么使用 `String&` ?
  - 能够保留原本的特性
  - 可以连续赋值
  - 可以作为左值
- 为什么不使用 `String` ?
  - 会产生临时对象
  - 会调用复制构造函数
  - 会调用析构函数
- 为什么不使用 `void` ?
  - 无法连续赋值
  - 无法作为左值

赋值与初始化

区别:

- 赋值：对象已经存在，给对象赋一个新的值
  - “一到一”
- 初始化：对象还不存在，用一个新的值创建一个对象
  - “零到一”

时机：

- 当声明并定义对象的时候，发生初始化;当执行赋值表达式时，发生赋值
- 初始化的时候，调用构造函数;执行赋值表达式时，调用赋值运算符重载
- 接受函数的参数以及函数的返回值时，这个对象也会被初始化，而不是赋值

类型转换重载

现在，我们有一个object是A类型的，我们希望它能够被当作B类型的使用，这时候就需要类型转换重载

其一般形式为

```
operator type() const {
        ...
}
```

例如，在C++中，有一个Complex类，我们希望它能够被当作double类型的使用

```
class Complex {
public:
        double real, imag;
        Complex(double r, double i): real{r}, imag{i} {}
        operator double() const { return real; }
};
```

注意：

- 类型转换运算符重载函数不能有返回值
- 类型转换运算符重载函数不能有形参
- 类型转换运算符重载函数必须是成员函数
- 类型转换时会自动调用类型转换运算符重载函数
- 如果加上 `explicit` 则只能用于显式类型转换

非成员的运算符重载

运算符重载为友元函数

一般情况下，将运算符重载为成员函数是比较好的选择，但有时候，重载为成员函数不能满足要求

例如，`+`运算符重载为成员函数时，左操作数必须是对象，右操作数可以是对象也可以是基本数据类型，但加法本身具有对称性，所以我们希望`a + b`和`b + a`是一样的，这时候就需要重载为非成员函数

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i): real{r}, imag{i} {}
        friend Complex operator+(const int n, const Complex& c);
        // 声明为友元，以便能够重载为普通函数的运算符能够访问private成员
};
Complex operator+(const int n, const Complex& c) {
        return Complex(n + c.real, c.imag);
}
```

还有没有其他方法?

重载为成员函数，但是补充类型转换构造函数，将int/double等类型转换为Complex类型

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i): real{r}, imag{i} {}
        Complex(int n): real{n}, imag{0} {}
        Complex(double r): real{r}, imag{0} {}
        Complex operator+(const Complex& c) {
                return Complex(real + c.real, imag + c.imag);
        }
};
```

流插入和流提取运算符的重载

cout是什么?

- cout是ostream类的对象
- ostream类重载了`<<`运算符

怎么重载才能既输出整数，又输出字符串?

- 函数重载

为什么能够连续使用`<<`?

- 返回值是ostream类的对象

现在考虑，我们如果想要重载 `<<` 与 `>>` 运算符，使得 `std::cin` 与 `std::cout` 能够输出我们自定义的Complex类的数据，应该怎么做？

考虑 `ostream` 类是已经写好的类，无法修改，我们需要将运算符重载为非成员函数

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i = 0): real{r}, imag{i} { }
        friend ostream& operator<<(ostream& os, Complex& c)
        friend istream& operator>>(istream& is, Complex& c)
};
ostream& operator<<(ostream& os, const Complex& c) {
        os << c.real << "+" << c.imag << "i";
        return os;
}
istream& operator>>(istream& is, Complex& c) {
        is >> c.real >> c.imag;
        return is;
}
```

特殊运算符的重载

自增减运算符

自增减运算符有前置(++i)和后置(i++)两种形式，前置形式返回的是自增减后的值，后置形式返回的是自增减前的值

为了区分重载的是哪个运算符，C++规定前置运算符作为一元运算符重载，后置运算符作为二元运算符重载

```
class Complex {
private:
        double real, imag;
public:
        Complex(double r, double i): real{r}, imag{i} {}
        Complex& operator++() {
                real++;
                imag++;
                return *this;
        }
        // 取回值为增加后的值，所以返回*this
        Complex operator++(int) {
                Complex c(*this);
                real++;
                imag++;
                return c;
```

```
        }
        // 取回值为增加前的值，所以返回临时对象
};
```

函数调用运算符

函数调用运算符是 `func(a, b)` 中的括号，一般来说，任何非函数的类型都没有定义其函数调用运算符的重载，我们无法将一个普通变量作为函数来用，但是重载了函数调用运算符就可以实现这个效果

```
class Complex {
        Complex operator()(parameters, ...) {
                return //类型任意
        }
};
```

函数调用运算符只能以成员函数的形式定义重载，不能以普通函数的形式定义重载

Note

- 正确性/鲁棒性
  - 预置赋值运算符重载无法实现深复制，需要手动编写来创建新的内存空间，避免后续修改对原有对象的影响
- 可扩展性
  - 运算符重载使得C++预定义的运算符可以支持自定义数据类型使用运算符来表示特定操作
- 可复用性
  - C++预定义的运算符可被自定义数据类型的对象复用
- 可理解性
  - 使用重载后的运算符比使用函数更容易清晰的理解对象之间的操作
  - 需注意重载后运算符的含义应该符合日常习惯，否则会造成误导

Lecture06-C++面向对象程序设计(4)继承

继承和派生

派生类的概念

在定义一个新的类B时，如果类B拥有某个已写好的 类A的全部特点，此外还有类A没有的特点，那么就 不必从头重写类B，而是可以

- 把类A作为一个基类(也称父类)
- 把类B作为类A的一个派生类(也称子类)

派生类是通过对基类进行扩充和修改得到的

- 扩充:在派生类中，可以添加新的成员变量和成员函数
- 修改:在派生类中，可以重新编写从基类继承 得到的成员

派生类的写法

C++中，从一个类派生出一个新的类的语法格式如下

```
class DerivedClassName : access-specifier BaseClassName {
        ...
};
```

其中，继承方式说明符有：

- private
- public
- protected

注意：

- 派生类拥有基类的所有成员
- 派生类的各个成员函数不能访问基类的私有成员
- 派生类的成员函数可以访问基类的保护成员

示例

```
class Student {
private:
        std::string sName;
        int gender
public:
        bool isThreeGood();
        void setName(const std::string& name);
};

class Undergraduate: public Student {
private:
        int department;      // 扩充
public:
        bool isThreeGood();  // 修改
        bool canBaoYan();    // 扩充
};
```

成员的覆盖

在派生类中，可以定义和基类成员同名的成员

- 在派生类中访问与基类同名的成员时，会优先访问派生类的成员，缺省访问派生类中定义的成员
- 在派生类中访问由基类继承来的成员时，可以使用作用域解析运算符`::`来指明访问的是基类的成员

```
class Base {
        int j;
public:
        int i;
        void func();
};

class Derived: public Base {
public:
        int i;                  // 覆盖
        void func();    // 覆盖
        void access();  // 扩充
};

void Derived::func() {
        j = 20                  // error，不能访问基类的私有成员
        i = 10;                 // 访问Derived的i
        Base::i = 20;   // 访问Base的i
        Base::func();   // 访问Base的func
}
```

派生类的成员组成与可见性

公有继承：

让基类的所有公有成员都可被外部访问，即基类的公有成员在派生类中也是公有的

私有继承：

让基类的所有公有成员都不可被外部访问，即基类的公有成员在派生类中是私有的

在公有继承中

- 基类的私有成员，可以被下列函数访问
  - 基类的成员函数
  - 基类的友元函数
- 基类的公有成员，可以被下列函数访问
  - 基类的成员函数
  - 基类的友元函数
  - 派生类的成员函数

    - 派生类的友元函数
    - 其他的函数
- “基类的私有成员对派生类不可见”的设定有时不够灵活，需要有一种折中的机制
    - 基类的成员可以被派生类访问，但又不对外公开

用protected修饰的成员称为保护成员， 保护成员可被访问的范围比私有成员大，比公有成员小

- 不考虑继承，一个类的保护成员可访问范围等同于私有成员
- 在公有继承中，基类的保护成员仅可以被下列函数访问
    - 基类的成员函数
    - 基类的友元函数
    - 派生类的成员函数可以访问当前对象的基类的保护成员，也可以访问其它同类对象的基类的保护成员
    - 派生类的友元函数

```
class Father {
private:
        int nPrivate;
public:
        int nPublic;
protected:
        int nProtected;
};

class Son: public Father {
public:
        void AccessFather() {
                nPrivate = 1;    // error
                nPublic = 2;     // ok
                nProtected = 3; // ok
                Son s;
                s.nProtected = 4;        // ok
                Father f;
                f.nProtected = 5;        // error
        }
};
```

Important

- 公有继承 (public)
    - 基类的公有成员和保护成员作为派生类的成员时，它们都保持原有的状态，基类的私有成员对派生类不可见
- 私有继承 (private)

        - 基类的公有成员和保护成员都作为派生类的私有成员，基类的私有成员对派生类不可见
    - 保护继承 (protected)
        - 基类的所有公有成员和保护成员都成为派生类的保护成员，并且只能被它的派生类成员函数或友元访问，基类的私有成员对派生类不可见

派生类的构造和析构

- 基类的构造函数是不会被继承的
- 可访问的（公有，非=delete）的基类的复制赋值运算符重载，也不会被继承
- 基类的析构函数不会被继承的

```
class Bug {
private:
        int nLegs;
        int nColor;
public:
        int nType;
        Bug(int legs, int color): nLegs{legs}, nColor{color} { }
        void PrintBug();
};

class FlyBug : public Bug {
        int nWings;
public:
        FlyBug(int legs, int color, int wings);
};

FlyBug::FlyBug(int legs, int color, int wings) {
        nLegs = legs;   // error，基类的私有成员不可访问
        nColor = color; // error，基类的私有成员不可访问
        nWings = wings; // ok
}

FlyBug::FlyBug(int legs, int color, int wings): Bug(legs, color), nWings{wings} {
}
// 通过初始化列表调用基类的构造函数
```

Important

- 在创建派生类的对象时，需要调用基类的构造函数
    - 初始化派生类对象中从基类继承的成员
    - 在执行派生类的构造函数前，总是先执行基类的构造函数
- 派生类构造顺序

    - 构造基类部分的成员:如果初始化列表中提供了基类构造方法，则调用对应构造函数；否则默认初始化基类;
  - 按照初始化列表或者默认初始化器对派生类“额外的”成员进行构造。若无，则默认初始化;
  - 执行派生类的构造函数
  - 派生类的析构顺序一般和其构造顺序是相反的
    - 执行析构函数
    - 析构成员:顺序与构造(初始化)相反
    - 执行基类的析构函数

```
class Base {
public:
        int n;
        Base(int i): n{i} {
                std::cout << "Base" << n << "constructed" << std::endl;
        }
        ~Base() {
                std::cout << "Base" << n << "destructed" << std::endl;
        }
};

class Derived : public Base {
public:
        Derived(int i): Base(i) {
                std::cout << "Derived constructed" << std::endl;
        }
        ~Derived() {
                std::cout << "Derived destructed" << std::endl;
        }
};
```

包含成员对象的派生类的构造函数

```
class Bug {
private:
        int nLegs;
        int nColor;
public:
        int nType;
        Bug(int legs, int color): nLegs{legs}, nColor{color} { }
        void PrintBug();
};

class Skill {
public:
```

```
        Skill(int n) { }
};

class FlyBug : public Bug {
        int nWings;
        Skill sk1, sk2;
public:
        FlyBug(int legs, int color, int wings): Bug(legs, color), sk1{5},
sk2{color} {
                nWings = wings;
        }
};
```

继承中的隐式转换

派生类对象到基类对象的转换

一个公有继承的派生类对象可以隐式转换到其基类对象

- 转换后的对象不包含派生类定义的成员，因为没有派生类成员的存储空间(派生类部分在赋值时被“切掉”)
- 从派生类构造基类时会调用基类的复制构造(相当于从基类构造基类)
- 将派生类赋值到基类会调用基类的复制赋值重载

```
class Base { };
class Derived : public Base { };
void f(Base b);

int main() {
        Derived d;
        Base b{d};  // ok
        b = d           // ok
        f(d)            // ok
}
```

基类引用绑定到派生类

若派生类公有继承自基类，则基类引用可绑定到派生类对象

- 虽然这个引用绑定到了派生类，但它仍然只能访问基类的成员数据和成员函数

```
class Base {
public:
    int baseMem;
};

class Derived : public Base {
```

```
public:
    int derivedMem;
};

int main() {
    Derived d;
        Base& b{d};
        b.baseMem;         // OK，访问基类成员
        b.derivedMem; // 错误，访问派生类成员
}
```

基类与派生类的指针强制转换

若派生类公有继承自基类，则基类指针可以直接指向派生类对象

```
Base* ptrBase = &objDerived;
```

其中，

- `ptrBase` 指向的是一个 `Derived` 类的对象 `objDerived`
- `*ptrBase` 可以看作一个 `Base` 类的对象，可使用 `ptrBase` 访问 `Base` 的公有成员
- 即便基类指针指向的是一个派生类的对象，也不能通过基类指针访问基类没有、而派生类中有的成员
    - 不能通过 `ptrBase` 访问 `objDerived` 对象中属于 `Derived` 类而不属于 `Base` 类的成员
- 通过强制指针类型转换，可以把ptrBase转换成Derived类的指针
    - 需保证ptrBase指向的是Derived类的对象，否则会有问题

```
Derived* ptrDerived = (Derived*)ptrBase;
```

```
class Base {
protected:
    int n;
public:
    Base(int i) : n(i) {
                cout << "Base " << n << " constructed" << endl;
        }
        ~Base() {
                cout << "Base " << n << " destructed" << endl;
        }
        void print() {
                cout << "Base:n=" << n << endl;
        }
};

class Derived : public Base {
```

```
public:
        int v;
        Derived(int i) : Base(i), v(2 * i) {
                cout << "Derived constructed" << endl;
        }
        ~Derived() {
                cout << "Derived destructed" << endl;
        }
        void func() {}
        void print() {
                cout << "Derived:v=" << v << endl;
                cout << "Derived:n=" << n << endl;
        }
};

int main() {
    Base objBase(5);
        Derived objDerived(3);
        Base *pBase = &objDerived;
        //pBase->func(); //错误，Base类没有func()成员函数
        //pBase->v = 5; //错误，Base类没有v成员变量
        pBase->print(); //调用基类函数print()
        //Derived *pDerived = &objBase; //错误
        Derived *pDerived = (Derived *)(& objBase);
        pDerived->print(); //慎用，可能出现不可预期的错误
        //往别人的空间里写入数据，可能引起问题
        pDerived->v = 128;
        objDerived.print();
        return 0;
}
```

Important

- 派生类的对象可以赋值给基类对象
  - `b = d;`
- 派生类对象可以初始化基类引用
  - `Base &br = d;`
- 派生类对象的地址可以赋值给基类指针
  - `Base *pb = &d;`

多继承

直接基类与间接基类

- 类A派生类B，类B可再派生类C，类C派生类D，……

    - 类A是类B的直接基类
    - 类B是类C的直接基类，类A是类C的间接基类
    - 类C是类D的直接基类，类A, B是类D的间接基类
- 在声明派生类时, 派生类的首部只需要列出它的直接基类
    - 派生类的首部不要列出它的间接基类
    - 派生类沿着类的层次自动向上继承它的间接基类
    - 派生类的成员包括
        - 派生类自己定义的成员
        - 直接基类中定义的成员
        - 全部间接基类的成员

多继承

一个类可以由多个基类派生而来，以继承多个基类的成员

```
class Derived : access-specifier1 Base1, access-specifier2 Base2, ... {
        ...
};
```

多继承的优点

- 可以从多个基类继承不同的特性
- 可以在派生类中定义新的成员，以扩充和修改继承的成员

多继承的缺点

- 可能会导致派生类对象中包含多个基类对象的副本
- 调用时可能会出现二义性问题

```
class Base1 {
private:
        int b1;
        void set(int&);
public:
        int i;
};

class Base2 {
private:
        int b2;
public:
        void set(int&);
        int get();
        int i;
};
```

```
class Derived : public Base1, public Base2 {
public:
        void print() {
                std::cout << get();
                set(5);                  // error，二义性
                Base1::set(5);   // error，private函数
                Base2::set(5);   //     ok
        }
};
```

对象之间的关系

- 面向对象将现实世界中的概念抽象为“类”，而把符合这些概念的事物抽象为“对象”
- 现实世界中，两个对象之间可能存在各种各样的关系
  - 泛化:分类法中一般和特殊的关系
    - 是(is-a) :大学生是学生
  - 关联:静态结构上的关系，长期的、稳定的
    - 有(has-a):汽车有轮子
    - 部分于(part-of):例如头是人体的一部分
  - 依赖:动态行为上的关系，偶发的、临时性的
    - 使用(uses-a):程序员使用键盘
    - 依赖于(depends-a):花依赖于蜜蜂传粉
- 具体的关系和使用场景有关，需视情况分析

Important

- 使用公有继承机制可以实现“是”关系
  - 私有继承和保护继承一般不能表示泛化关系
- 使用类的成员变量可以实现关联关系
  - 成员变量使用类类型对象可以表示“部分于”关系，UML中称为“组合”关系
  - 成员变量使用类类型的指针可以表示“有”关系，UML中称为“聚合”关系
- 使用类的成员函数参数可以实现依赖关系

Note

- 正确性/鲁棒性
  - 构造函数不会被继承，可防止派生类部分成员的未初始化状态
  - 不同的继承方式可以实现不同程度的信息隐藏，防止因 继承暴露更多信息
- 可扩展性
  - 通过派生可以无需修改基类实现即可增加特殊功能
- 可复用性

        - 继承机制可以复用基类的代码
    - 可理解性
        - 可以使用作用域符号指明使用基类还是派生类的成员
        - 在基类和派生类中一般不使用同名成员变量，以免造成 歧义
        - 多继承可能造成二义性，慎用，可以转化为单继承

Lecture07-C++面向对象程序设计(5) 多态

虚函数和多态

在类的定义中，前面有virtual关键字的成员函数就是虚函数

- virtual关键字只需在类定义里的函数声明中写
- 在类外定义虚函数的时候不需要写virtual关键字
- 静态成员函数不能是虚函数

多态的表现形式1：指针的多态

- 基类类型的指针可以指向基类类型的对象，也可以指向派生类类型的对象
- 当通过基类指针调用基类和派生类中的同名虚函数时
    - 若指针指向一个基类的对象，则被调用的是基类的虚函数
    - 若指针指向一个派生类的对象，则被调用的是派生类的虚函数

多态：根据指针指向的对象类型来决定调用的函数

示例：

```
class Base {
public:
        virtual void func() {
                std::cout << "Base::func()" << std::endl;
}
};

class Derived : public Base {
public:
        virtual void func() {
                std::cout << "Derived::func()" << std::endl;
        }
};

int main() {
        Derived oDerived;
        Base* pBase = &oDerived;
        pBase->func();  // Derived::func()
```

```
        return 0;
}
```

多态的表现形式2：引用的多态

- 基类类型的引用可以引用基类类型的对象，也可以引用派生类类型的对象
- 当通过基类引用调用基类和派生类中的同名虚函数时
    - 若引用引用一个基类的对象，则被调用的是基类的虚函数
    - 若引用引用一个派生类的对象，则被调用的是派生类的虚函数

多态：根据引用绑定的对象类型来决定调用的函数

```
int main() {
        Derived oDerived;
        Base& rBase = oDerived;
        rBase.func();   // Derived::func()
        return 0;
}
```

虚函数的实现原理（by Github Copilot)

- 虚函数的实现原理是通过虚函数表来实现的
- 每个类有一个虚函数表，虚函数表中存放晛类的虚函数地址
- 每个对象有一个虚函数指针，指向类的虚函数表
- 调用虚函数时，通过虚函数指针找到虚函数表，再通过虚函数表找到虚函数的地址，最后调用虚函数

虚函数性质的传递

任何用于覆盖虚函数的派生类同名函数，也是虚函数（要求签名必须完全相同）

- 无论在这些派生类的同名函数前是否显式加virtual

```
 class A {
 public:
        virtual void f() { }
 };

 class B : public A {
 public:
        void f() { }   //尽管没有virtual关键字修饰，但它仍然是虚函数
};

 class C : public B {
        void f() { }   //C::f()也是虚函数，因为B::f()是虚函数
};
```

```
int main() {
        A* a;
        B* b;
        b = new C;
        b->f(); //调用 C::f() 而非 B::f()
        a = b;
        a->f(); //调用 C::f() 而非 A::f()
        delete a;
}
```

override

然而，让程序员自己注意两个函数签名是否相同是可能犯错的

1. 忘了 `const`
2. `Destroy` 写成 `De-story` 了（
    - 如果不加 `virtual`，编译通过，多态只会调用到A自己的函数；
    - 加了 `virtual`，编译还是通过，甚至产生了一个新的虚函数。

- 解决方案：`override`！
    - 除非函数签名与某个基类的虚方法完全一致，否则编译器会报错。

```
class A {
public:
        virtual void DestroyIt() const;
};

class B : public A {
public:
        void DestroyIt();
};

class B : public A {
public:
        void DestroyIt() override;
}; // 报错
```

final

`final` 也具有 `override` 的功能，但是他同时进行了加强；他的子类不能再覆盖这个函数，后续子类也只能使用当前类所提供的虚函数

```
class B : public A {
public:
        void DestroyIt() const final;
```

```
};

class C : public B {
public:
        void DestroyIt() const; // 报错
};
```

类也可以用`final`修饰，表示这个类不能被继承

```
class B final : public A {
public:
        void DestroyIt() const final;
};

class C : public B { }; // 报错
```

基类的普通成员函数中调用虚函数也会有多态

- 相当于隐含了基类类型指针this

```
class Base {
public:
        void callF() {
                f(); // 相当于this->f(); this是基类指针
        }
        virtual void f() { //虚函数，可被子类覆盖
                cout<<"Base called"<<endl;
        }
};

class Derived : publicBase {
public:
        void f() { cout<<"Derived called"<<endl; }
};

int main() {
        Base* b= new Derived;
        b->callF(); // 输出Derived called
}
```

构造函数和析构函数中调用虚函数

在基类的构造函数和析构函数中调用虚函数时，调用的是自己的类（这个被调用的类）中定义的函数（此时没有多态）

- 因为在运行构造函数和析构函数的时期，派生类尚未形成或已经消失

```cpp
class Father {
public:
        virtual void hello() {
                cout << "hello from Father" << endl;
        }
        virtual void bye() {
                cout << "bye from Father" << endl;
        }
};

class Son : public Father {
public:
        void hello() {
                cout << "hello from Son" << endl;
        }
        Son() { hello(); }
        ~Son() { bye(); }
};

class Grandson : public Son {
public:
        void hello() {
                cout << "hello from Grandson" << endl;
        }
        Grandson() { cout << "constructing Grandson" << endl; }
        ~Grandson() { cout << "destructing Grandson" << endl; }
};

int main() {
        Grandson g;
        Son* pson;
        pson = &g;
        pson->hello();
        return 0;
}

// hello from Son
// constructing Grandson
// hello from Grandson
// destructing Grandson
// bye from Father
```

虚函数的访问权限

虚函数的访问权限可以是public、protected或private

其访问权限检查根据指针类型来判断

```
class Base {
private:
        virtual void fun2() { cout << "Base::fun2()" << endl; }
};

class Derived : public Base {
public:
        virtual void fun2() { cout << "Derived::fun2()" << endl; }
};
int main() {
        Derived d;
        Base* pb = &d;
        pb->fun2(); // error, Base::fun2是private
}
```

如果将 `Base` 类中的 `fun2` 函数声明为 `public`，则可以正常调用（即使 `Derived` 是 `private`）

虚析构函数

下面程序输出什么?

```
class Son {
public:
        ~Son() { cout << "Son destructed" << endl; }
};

class Grandson : public Son {
public:
        ~Grandson() { cout << "Grandson destructed" << endl; }
};

int main() {
        Son* p;
        p = new Grandson;
        delete p;
        return 0;
}
// Son destructed
```

没有调用 `Grandson::~Grandson()`，因为 `Son::~Son()` 不是虚函数

注意:

- 通过基类的指针删除派生类对象时，通常情况下只调用基类的析构函数
- 但是删除一个派生类的对象时，应该先调用派生类的析构函数，然后调用基类的析构函数
    - 特别是派生类中有动态分配的内存需要在析构函数中释放

- 解决办法：把基类的析构函数声明为 `virtual`
  - 派生类的析构函数可以不声明为 `virtual`，但是建议也声明为 `virtual`
  - 通过基类的指针删除派生类对象时，首先会调用派生类的析构函数，然后再调用基类的析构函数
  - 一般来说，一个类如果定义了虚函数，则应该将析构函数也定义成虚函数
- 注意：构造函数不能作为虚函数

修改上方示例：

```
class Son {
public:
        virtual ~Son() { cout << "Son destructed" << endl; }
};

class Grandson : public Son {
public:
        ~Grandson() { cout << "Grandson destructed" << endl; }
};

int main() {
        Son* p;
        p = new Grandson;
        delete p;
        return 0;
}
// Grandson destructed
// Son destructed
```

纯虚函数与抽象类

纯虚函数

纯虚函数：带有纯说明符 `=0` 的虚函数

```
class A {
private:
        int a;
public:
        virtual void print() = 0;
        void fun() {
                cout << "fun";
        }
}

void A::print() {
```

```
        cout << "A::print()";
}
```

- 由于语法限制，纯说明符=0和函数体无法同时出现，所以纯虚函数不允许类内定义
- 如果一个纯虚函数始终不会被调用，就无需给出它的定义
- 可以为纯虚函数提供类外定义
    - 如果纯虚函数是析构函数就必须提供定义

抽象类

包含纯虚函数的类叫抽象类

- 抽象类不能用于创建对象
- 抽象类只能作为基类来派生新类使用
- 抽象类的指针和引用可以指向由抽象类派生出来的类的对象

```
A a; // error，抽象类不能创建对象
A* pa; // ok，抽象类的指针
pa = new A; // error，抽象类不能创建对象
A& ra = a; // ok，抽象类的引用（引用子类）
```

在抽象类的成员函数内可以调用纯虚函数

- 但在构造函数或析构函数内部不能调用纯虚函数

示例：

```
class A {
public:
        virtual void f() = 0;
        void g() { this->f(); }
        A() { } // 如果 A() { f(); } 则会报错
};

class B : public A {
public:
        void f() { cout << "B::f()" << endl; }
};

int main() {
        B b;
        b.g();
        return 0;
}
```

抽象类的作用

- 如果一个类从抽象类派生而来，那么当且仅当它实现了基类中的所有纯虚函数，它才能成为非抽象类
- 抽象类经常作为一个一般性概念而存在；它会包含若干个具体的派生类作为这个一般性概念的具体解释。抽象类的纯虚函数则作为一个约束，要求其派生类必须实现这些纯虚函数的定义

多态与可扩展性

- 多态的实质
  - 父类定义共同接口，子类不同实现
  - 通过父类以相同的方式操作不同子类的行为
- 在面向对象的程序设计中使用多态
  - 能够增强程序的可扩展性
  - 程序需要修改或增加功能的时候，需要改动和增加的代码较少

示例：《魔法门之英雄无敌》

游戏中有很多种怪物

- CSoldier
- CPhoenix
- CDragon
- CAngel

每种怪物都有一个类与之对应，每个怪物都是这个类的一个对象

怪物之间能够互相攻击，攻击敌人和被攻击时都有相应的动作，动作是通过对象的成员函数实现的

> **Question**
>
> 游戏版本升级时，要增加新的怪物——雷鸟，如何编程能使得升级时的代码改动和增加量较小？

基本思路：

- 为每个怪物类编写 `Attack`，`FightBack` 和 `Hurted` 等成员函数
  - `Attack`：表现攻击动作，攻击某个怪物
    - 调用被攻击怪物的 `Hurted` 函数，以减少被攻击怪物的生命值
    - 调用被攻击怪物的 `FightBack` 成员函数，遭受被攻击怪物反击
  - `FightBack`：表现反击动作，并调用被反击对象的 `Hurted` 成员函数，使被反击对象受伤
  - `Hurted`：减少自身生命值，并表现受伤动作

非继承的实现方法

- 有n种怪物，以`CDragon`举例，需要编写n个Attack成员函数以及n个FightBack和成员函数

```
class CDragon {
private:
        int nPower;
        int nLifeValue;
public:
        void Attack(CWolf* pWolf) {
                pWolf->Hurted(nPower);
                pWolf->FightBack(this);
        }
        void Attack(CGhost* pGhost) {
                pGhost->Hurted(nPower);
                pGhost->FightBack(this);
        }
        void FightBack(CWolf* pWolf) {
                pWolf->Hurted(nPower / 2);
        }
        void FightBack(CGhost* pGhost) {
                pGhost->Hurted(nPower / 2);
        }
        void Hurted(int nPower) {
                nLifeValue -= nPower;
        }
};
```

如果游戏版本升级，增加新的怪物雷鸟`CThunderBird`，则所有的类都需要增加`Attack`与`FightBack`成员函数

- 在怪物种类多时候，工作量很大

使用继承但不使用多态的实现方法

- 将`CDragon`、`CWolf`等类的共同特点抽取出来CCreature类
- 然后再从`CCreature`类派生出`CDragon`、`CWolf`等类
- 但是由于每种怪物进行攻击、反击和受伤时的表现动作不同；当增加新的怪物时，仍然需要在每个已有的类里面增加相应针对新怪物的`Attack`、`FightBack`成员函数
- 如果没有利用多态机制，即便引入基类`CCreature`，对程序的可扩展性也无帮助

使用多态的实现方法

- 当增加新怪物雷鸟的时候，只需要编写新类`CThunderBird`，不需要在已有的类里专门为新怪物增加成员函数
- 已有的类可以原封不动

```
class CCreature {
protected:
        int m_nLifeValue;
        int m_nPower;
public:
        virtual void Attack(CCreature* pCreature) = 0;
        virtual void FightBack(CCreature* pCreature) = 0;
        virtual void Hurted(int nPower) = 0;
};

class CDragon : public CCreature {
public:
        virtual void Attack(CCreature* pCreature) {
                pCreature->Hurted(m_nPower);
                pCreature->FightBack(this);
        }
        virtual void FightBack(CCreature* pCreature) {
                pCreature->Hurted(m_nPower / 2);
        }
        virtual void Hurted(int nPower) {
                m_nLifeValue -= nPower;
        }
};
```

示例：几何体处理程序

- 输入若干个几何形体的参数，要求按面积排序输出，并且输出时要指明形状
  - 输入：第一行是几何形体数目n (不超过100)，下面有n行，每行以一个字母c开头
  - 若c是R，则代表一个矩形，本行后面跟着两个整数，分别是矩形的宽和高
  - 若c是C，则代表一个圆，本行后面跟着一个整数代表其半径
  - 若c是T，则代表一个三角形，本行后面跟着三个整数，代表三条边的长度
  -按面积从小到大依次输出每个几何形体的种类及面积，每行一个几何形，格式为 形体名称:面积

输入示例	输出示例
3 R 3 5 C 9 T 3 4 5	Triangle:6 Rectangle:15 Circle:254.34

类的设计

```
class CShape {
public:
        virtual double Area() = 0;
        virtual void PrintInfo() = 0;
```

```
}; // CShape是抽象类

class CRectangle : public CShape {
public:
        double w, h;

        virtual double Area() { return w * h; }
        virtual void PrintInfo() { cout << "Rectangle:" << Area() << endl; }
}; // CRectangle是具体类

class CCircle : public CShape {
public:
        double r;

        virtual double Area() { return 3.1415 * r * r; }
        virtual void PrintInfo() { cout << "Circle:" << Area() << endl; }
}; // CCircle是具体类

class CTriangle : public CShape {
public:
        double a, b, c;

        virtual double Area() {
                double p = (a + b + c) / 2.0;
                return sqrt(p * (p - a) * (p - b) * (p - c));
        }
        virtual void PrintInfo() { cout << "Triangle:" << Area() << endl; }
}; // CTriangle是具体类
```

主程序

```
int main() {
        int n;
        CShape* pShape[100];
        cin >> n;
        for (int i = 0; i < n; i++) {
                char c;
                cin >> c;
                switch (c) {
                        case 'R':
                                pShape[i] = new CRectangle;
                                cin >> ((CRectangle*)pShape[i])->w >>
((CRectangle*)pShape[i])->h;
                                break;
                        case 'C':
                                pShape[i] = new CCircle;
                                cin >> ((CCircle*)pShape[i])->r;
                                break;
```

```
                        case 'T':
                                pShape[i] = new CTriangle;
                                cin >> ((CTriangle*)pShape[i])->a >>
((CTriangle*)pShape[i])->b >> ((CTriangle*)pShape[i])->c;
                                break;
                }
        }
        for (int i = 0; i < n; i++) {
                pShape[i]->PrintInfo();
        }
        return 0;
}
```

排序函数

```
void sort(CShape** shapes, int n) {
        for (int i = 0; i < n; ++i) {
                for (int j = 0; j < i; ++j) {
                        double a1 = shapes[i]->Area();
                        double a2 = shapes[j]->Area();
                        if (a1 < a2) {
                                CShape* temp = shapes[i];
                                shapes[i] = shapes[j];
                                shemp[j] = temp;
                        }
                }
        }
}
```

Question

如果要增加新的几何形体，如何修改程序?

只需要从 `CShape` 派生一个新的类，实现 `Area` 和 `PrintInfo` 成员函数，在 `main` 函数的 `switch` 语句中增加一个新的 `case` 即可

Note

常用技巧：使用基类指针数组存放指向各种派生类对象的指针，然后遍历该数组，就能对各个派生类对象做各种操作

运行时类型识别（RTTI）

Question

> 当派生类有自己独有的成员函数时，如何在运行时动态调用？

给定一个类：

```
class Animal {
public:
        virtual string getName() { return "Animal"; }
};

class Cat : public Animal {
public:
        string getName() { return "Cat"; }
        void meow() { }
};

class Dog : public Animal {
public:
        string getName() { return "Dog"; }
        void bark() { }
};
```

如果想要编写这样一个功能：检查一个`Animal`是不是`Dog`，如果是的话就让它`bark()`，否则什么都不做。怎么实现呢？

一种做法：

```
void tryBark(const Animal* a) {
        if (a->getName() == "Dog") {
                const Dog* d = (const Dog*)a;
                d->bark();
        }
}
```

这样做有什么问题？

- 依赖于一个不可靠的自定义函数getName，必须为每一个Animal 的子类定义一个getName虚函数的覆盖，并且不同子类的返回值必须不同，维护成本高

解决方案：

- typeid运算符
- dynamic_cast运算符

注意：C++只允许拥有虚函数的类使用RTTI

typeid运算符

使用 `typeid` 运算符可以在运行时确定一个对象的类型

- typeid(表达式)或typeid(类型名)
  - 返回一个type_info对象，这个对象包含了表达式的类型信息
  - type_info是一个类，定义在头文件typeinfo中
  - type_info对象可以用来比较两个类型是否相同
  - 对于相同“表达式”的类型或相同“类型”，typeid运算符总是得到相同的值（即== 为真）
  - 可以得到派生类对象的运行时“实际”类型

```
#include <iostream>
#include <typeinfo>
using namespace std;

class B {
public:
        virtual ~B() { }
};

class D1 : public B { };
class D2 : public B { };

int main() {
        B* pb = new D1;
        if (typeid(*b) == typeid(D1)) {
                cout << "b has type D1" << endl;
        }
        if (typeid(*b) == typeid(D2)) {
                cout << "b has type D2" << endl;
        }
        delete b;
}
```

上述检查 `Animal` 是不是 `Dog` 的函数可以改写为：

```
void tryBark(const Animal* a) {
        if (typeid(*a) == typeid(Dog)) {
                const Dog* d = (const Dog*)a;
                d->bark();
        }
}
```

多态类型

- 拥有虚函数的类及其派生类可被称为多态类型
- 多态类型的指针（或引用）会指向（或绑定到）两种意义上的类型：
  - 一是声明时所使用的类型
    - 类型在编译期间就可以确定，称为静态类型
  - 二是所“实际”指向的类型
    - 该类型会因为每一次运行随着用户输入变化可能得到不同的类型，称为动态类型
- 静态类型和动态类型只存在于多态类型指针（或引用）所指向（或绑定到）类型这一范畴内

补充：C++风格的类型转换

- 大家在之前使用的类型转换一直是C风格的，像 `(int)1.0` 这种
  - 或者隐式转换，例如函数类型到函数指针的隐式转换（因此可以不对函数取地址就赋给函数指针）、各种整数和浮点数的转换、子类指针或引用到父类指针或引用的转换，等等
- 但是C风格转换所囊括的内容太多了，把一些不够安全的转换也给杂糅进去了
  - 比如定义了一个 `const int a`，我们强行把 `&a` 转换为 `int*`，去掉其底层的 `const` 属性，再做修改，这可能是UB（未定义行为）
  - 或者定义了一个 `int a`，我们把 `&a` 转换为 `float*` 再读出来；`float` 并不在它的生命周期中，那里是个 `int`，因此这仍然是UB
    - gcc看到这个UB，在一些情况下真的会直接给优化掉

为了区分安全的和危险的转换，分清楚转换的职责，C++分解出了四种显式转换的关键字

- 语法：xxx_cast<目标类型>（想要转换的表达式）
- `static_cast`：基本安全的转换，涉及指针或引用的操作可能要注意一下安全性
- `dynamic_cast`：非常安全的转换，专用于继承与多态，开销稍微大一些
- `const_cast`：比较危险的转换，需要谨慎使用
- `reinterpret_cast`：非常危险的转换，需要程序员仔细思考它的功能

static_cast

`static_cast` 有以下几个主要功能：

- 缩窄转换，即从一个表示范围较大的类型转换为一个较小的类型
  - 例如将一个 `double` 转为 `int` 等，可以 `static_cast<int>(d)` 等等
  - 注意有符号数到无符号数的转换也被视为缩窄转换，因为前者能表示负数，后者不能
- 枚举的转换
- 将基类指针或引用转为派生类指针或引用
  - 注意：必须保证基类指针指向的确实是那个派生类的对象；例如 `Base* ptr = new Base`，随后 `Derived* ptr2 = static_cast<Derived*>(ptr)`，是不能用 `ptr2` 对成员

进行访问的；它内部没有派生类对象
- 与 `void*` 的转换等等，这门课不会使用

dynamic_cast

可以注意到，基类和派生类的转换要求程序员确保一定的条件

- 有没有什么方法，可以自动检测这个条件，不满足时报告给程序员呢？
- 这就是 `dynamic_cast` 的功能
  - 具有虚函数的类可以进行运行时类型识别（RTTI），从而帮助程序员确定类型信息
  - 当基类指针使用 `dynamic_cast` 转为派生类指针时，如果它内部的对象不是这个类型的派生类，那么会返回 `nullptr`
    - 例如对继承关系A->B->C->D，如果当前对象是C类对象，而我们拥有一个 `A* ptr`，则 `dynamic_cast<B*>(ptr)`, `dynamic_cast<C*>(ptr)` 都会得到正确的指针，而 `dynamic_cast<D*>(ptr)` 会得到 `nullptr`
  - 对于错误的引用转换会抛出异常，我们后面的课程会讲什么是异常

const_cast

去掉指针或者引用的底层 `const` 属性

- 通常出现在一个库的函数的某个参数忘记了 `const` 属性，而事实上代码并没有对它做修改，此时我们可以把手里的 `const A* ptr` 强行通过 `const_cast<A*>(ptr)` 提供给这个函数
  - 如果内部是 `const` 对象，我们转型后对其修改一般是UB，结果未知

例如：

```
const int a = 1;
int* b = const_cast<int*>(&a);
*b = 2; // UB
```

reinterpret_cast

各种指针和引用的随意转换、指针与整数的相互转换。

- 如何安全地使用它是非常复杂的，一般情况下不建议使用
- 例如 `int a = 1; int* b = reinterpret_cast<int*>(&a);`，这样的转换是非常危险的

`reinterpret_cast` 使用方法：（by Github Copilot）

- `reinterpret_cast<type>(expression)`
- 用于位的重新解释，可以将一个指针转换为一个整数，也可以将一个整数转换为一个指针
- 用于不同类型的指针之间的转换，如将一个 `int` 指针转换为一个 `char` 指针

- 用于不同类型的引用之间的转换，如将一个 `int` 引用转换为一个 `char` 引用
- 用于不同类型的指针和引用之间的转换，如将一个 `int` 指针转换为一个 `char` 引用
- 用于不同类型的函数指针之间的转换，如将一个 `void(*)()` 函数指针转换为一个 `int(*)()` 函数指针
- 用于不同类型的对象之间的转换，如将一个 `int` 对象转换为一个 `char` 对象

补充：移动语义

移动语义主要有两个作用：

- 减少不必要的拷贝；
- 或者定义对数据的唯一所有权

我们不妨以一个自己定义的 `MyString` 类为例，假设我们已经实现了一个 `MyString`，现在我们要做如下操作

```
MyString a{"123"}, b{"456"}, c;
c = a + b;
```

考虑这个过程，我们首先对 `a`，`b` 做了构造，之后调用了 `operator+(a, b)`，那么，下一步呢？

下一步会调用 `MyString::operator=(const MyString&)`！

- `c` 于是析构掉自己的字符串、分配一个新的字符串，把 `a+b` 的返回结果拷贝过来

为什么我们不直接把 `a+b` 的结果的字符串“拿来”，这样就不必重新分配+拷贝+释放了呢？

- 也就是，我们把结果的指针直接作为当前的指针，而把原来的指针置空。
- 这一功能显然只能应用于临时值，也就是“纯右值”(prvalue)
- 或者虽然不是临时值，但我们明确地指出希望它被拿走，这也就是“亡值”(xvalue)。 •
- 二者合称为右值，这就是C++11引入这种值分类的原因。

比如现在 `d = "123456"`，我们想把 `d` 移动给 `c`，那我们可以直接让 `c` 指向 `"123456"`，接着令 `d = nullptr`

我们可以定义移动赋值函数来完成这一功能。
- 类似地还有移动构造函数
- `&&` 就是右值引用，左值并不能被绑定上去，因此不用担心一个变量的资源莫名其妙被拿走了。
- 如果你确实希望把一个左值的资源挪走， 你可以使用 `std::move` 将其转化为亡值，从而 `a = std::move(c);`

```
MyString(MyString&& another) : len{another.len}, ptr{another.ptr} {
        another.ptr = nullptr;
```

```
}
MyString& operator=(MyString&& another) {
        if (this == &another) {
                return *this;
        }
        delete[] ptr;
        ptr = another.ptr;
        another.ptr = nullptr;
        len = another.len;
        return *this;
}
```

- 特别地，被移动后的变量仍然存在，只是处于一种“被掏空”的状态，因此需要确保析构函数可以正确地处理这种空状态。
  - 我们这里 `delete[] ptr;` 可以正确处理 `nullptr`，所以没问题

是否还有其它问题？

- 其实我们类的一些不变量是被打破了的，例如它不再以\0结尾，也不再 能输出
  - `<<(const char*)nullptr` 是错误的
- 甚至我们前面的函数是没有对拿走的字符串的长度归零的，长度也没意义了

不过是否维持这些不变量并不是绝对的，所以一定要谨慎使用已经被移动过的变量——C++标准称其状态为valid but unspecified

- 一般来说（至少对标准库的类），只要对对象重新赋值，那么它就又回到了有效的状态，从而可以继续正常使用
- 这是由 `operator=` 中的内容决定的，我们上面写的类也有这个性质。

第二个作用就是定义对数据的唯一所有权

- 把复制构造函数和赋值函数给去掉，只保留移动构造函数和赋值函数。
- `A(const A&) = delete; A& operator=(const A&) = delete;`
- 我们在大作业中会用到智能指针 `std::unique_ptr`，它就是：
  - 在构造函数中得到分配的指针
  - 在析构函数中将其释放
  - 为了防止出现多次释放，只允许移动
  - 无深拷贝之语义，对 `new` 出来的内存进行唯一的拥有，也防止产生不必要的拷贝。

补充：值类别

表达式

- 表达式是某段内存及其相关操作（变量、运算、赋值）
- 类型是内存的解释方式

我们可以通过值类别对变量和内存进行精细的管理

- 设计目的
  - 尽量区分各种不同的表达式
  - 减少复制
  - 方便优化及特殊处理
- 一点点历史：
  - C++98：函数和对象的表达式叫左值，其他值叫右值
  - C++11：为了减少复制，引入了移动语义，重新定义了值类别
  - C++17：求进行复制消除，修改了右值的语义

考虑表达式的两种性质

- 是否拥有身份：
  - 可以通过某种方式唯一确定实体（身份证）
  - 满足该条件称为“泛左值”
- 能被移动
  - 所属内存能否被安全移动给别人
  - 满足该条件称为“右值”

左值：具有身份、不能被移动

例如，看如下代码

```
int main() {
        int x {42};
        int y {42};
}
```

- `x`和`y`的类型、值和类别都一样
- 但是`x += 1`与`y += 1`效果不同
- 因为`x`和`y`身份不同，它们是不同的内存名

纯右值：不具有身份、可被移动

```
50 + 2
52
```

- 两行表达式的类型、值和类别都一样
- 无法区分它们
- 不管哪个，用作初始化器/操作数的效果都是一样的
- 能被移动：这段内存编译器可以任意处理

C++17：关于纯右值的小变化

考虑下面代码中 `createVector()` 返回值的处理

- 这是一个纯右值：没有名字
- 但C++17规定，这里纯右值不允许被移动，必须原地实质化一个类
- 纯右值的定义改成了“可以实质化变量的值”

```
#include <vector?

std::vector<int> createVector() {
        return std::vector<int> {1, 2, 3, 4, 5};
}

int main() {
        std::vector<int> vec = createVector();
}
```

更多示例：

左值	右值
`a`	`42`
`a += b`	`false`
`a[n]`	`a + b`
`++a`	`a++`
`*p`	`&a`
返回引用的 `f()`	返回值的 `f()`

泛左值特性

- 通常可以取地址（`&a`）
  - 因为 `a` 拥有内存空间
- 通常可以被赋值（`a = ...`）
  - 相当于覆写其内存空间
- 可以绑定到左值引用上（`int& r = a;`）
  - 给这个内存空间再起一个名字
- 可以转换成纯右值
  - 泛左值具有纯右值的所有特性

只读左值引用 `const T&`：

- 左右通吃！
  - 既可以绑定到左值上，也可以绑定到右值上

- 绑定到右值时，临时量实质化得到的对象存储期延长到与 `refR` 相同

```
int main() {
        int a;
        const int& refL = a;
        const int& refR = 42;
}
```

> **Note**
>
> - 正确性/鲁棒性
> - 通过抽象类可以对其派生类的行为进行约束，要求其派生类必须实现抽象类中纯虚函数的定义
> - 使用虚析构函数以保证在多态时可调用到派生类的析构函数
> - RTTI可实现在运行时动态改变程序的行为，需小心使用
> - 可扩展性
> - 通过多态可以无需修改现有实现即可增加新类型
> - 可复用性
> - 通过多态可在新类型上复用原有已写好的功能
> - 可理解性
> - 不省略派生类虚函数的virtual关键字，以明确其性质

Lecture08-C++泛型程序设计(1) 模板与STL初步

考虑如下需求：

```
void printInt(int);
void printDouble(double);
void printString(string);
```

为了 `print` 不同类型的变量，我们写了很多不同的函数。如果我们想要实现一个函数，可以打印任意类型的数据，该怎么办呢？

可以使用函数重载！

```
void print(int);
void print(double);
void print(string);
```

但是这样的方法有很多缺点：

- 代码冗余

- 代码可读性差
- 代码可维护性差

我们还有没有其他的方法呢?

函数模板:

```
template <typename T>
void print(T t) {
        cout << t << endl;
}
```

泛型程序设计

- 泛型程序设计(Generic Programming)
  - 在编写代码时可以不指定具体的数据类型，而是使用抽象的类型来编写;实例化时再提供参数来指明类型
  - 优势:减少重复代码的编写，提高代码的可复用性
- C++泛型程序设计
  - 使用模板(Template)设计程序
    - 函数模板
    - 类模板
  - 标准模板库(Standard Template Library，STL)
    - 一些常用数据结构和算法的模板的集合

函数模板

函数模板是一种通用的函数描述，它用一个虚拟的类型T来代表函数的参数类型，实现了对不同类型的数据进行相同的操作

```
template <typename T>
ReturnType FuncName (parameters, ...) {
        // do_something
}
```

模板形参表是由一系列逗号分隔的模板形参

- 类型模板形参: typename 形参名 或 class 形参名
- 非类型模板形参
- 在函数模板的返回值类型、函数形参表、函数体中可以使用模板形参名来指代数据类型

从函数模板生成函数的过程称为函数模板的实例化

- 实例化发生在编译时

- 已实例化的函数被再次调用时不会重新被实例化

利用模板实参进行实例化

- 通过“带模板实参”的函数模板调用表达式来实例化函数模板

```
print<int>(42); //以int作为print模板的模板实参
```

- 当代码中出现了带模板实参的函数模板调用时，如果对应类型的函数尚未实例化，那么就开始进行一次模板实例化
  - 首先取出模板的声明，然后用模板实参列表中的实参替换声明中出现的模板形参
  - 替换完成后，整个声明就成为了模板的一份实例化函数，最终被调用的函数也正是这个实例化出来的函数

多个模板形参

```
template <typename T, typename U>
void add (T t, U u) {
        T c = t + u;
        cout << c << endl;
}

int main() {
        add<float, int>(1, 2);
}
```

我们一定要在模板形参列表中指定所有的模板实参吗?

不需要！编译器会根据函数实参的类型来推导模板实参的类型

函数模板实参推导(Function Template Argument Deduction，TAD)

在调用函数模板时可省略模板实参，而根据函数实参来推导模板实参的值

赋值兼容原则可能会导致二义性

```
template <typename T>
void myFunc(T arg1, T arg2) {
        cout << arg1 << " " << arg2 << endl;
}

myFunc(5, 7)            // ok, myFunc<int>
myFunc(1.2, 3.4)        // ok, myFunc<double>
myFunc(1, 3.2)          // error, 二义性，编译器不知道T是double还是int
```

解决方法：可以引入多个模板形参，或者使用强制类型转换

函数模板的重载

函数模板可以和普通函数一样重载

```
template <typename T>
void myFunc(T arg) {
        cout << "one argument" << endl;
}

template <typename T1, typename T2>
void myFunc(T1 arg1, T2 arg2) {
        cout << "two arguments" << endl;
}

int main() {
        myFunc(1);              // one argument
        myFunc(1, 2);   // two arguments
}
```

函数和模板的匹配顺序

- 先找一个参数完全匹配的普通函数
    - 非由模板实例化而得的函数
- 若找不到，则找一个参数完全匹配的由函数模板实例化出来的函数
- 若还没找到，则在没有二义性的前提下，再找一个参数经过自动类型转换后能够匹配的普通函数
- 上面的都找不到，则报错

```
template <typename T>
T max(T a, T b) {
    cout << "TemplateMax" <<endl;
        return 0;
}

template <typename T1, typename T2>
T max(T1 a, T2 b) {
    cout << "TemplateMax2" <<endl;
        return 0;
}

double max(double a, double b) {
        cout << "MyMax" << endl;
        return 0;
}
```

```
int main() {
        int i = 4, j = 5;
        max(1.2, 3.4);  // MyMax
        max(i, j);              // TemplateMax
        max(1.2, 3);    // TemplateMax2
        return 0;
}
```

类模板

类模板是一种特殊的类，它可以处理多种类型的数据。类模板是泛型编程的一部分，允许程序员编写对任何类型都有效的代码。

类模板的工作原理是，定义一个类，但是让类型作为参数。然后，当你创建这个类的对象时，你可以用实际的类型替换模板参数。

类模板的一个常见用途是实现数据结构，如数组、链表、栈、队列等，这些数据结构可以存储任何类型的数据。

语法：

```
template <typename T>
class ClassName {
        // do_something
};
```

在类模板的成员变量、成员函数中可以使用模板形参名来指代数据类型

```
template<typename T>
class CArray{
    T* ptrElement;
    int size;
public:
        CArray(int length);
        ~CArray();
        int len;
        void setElement(T arg, int index);
        T getElement(int index);
};
```

用类模板创建对象时，需要按照如下语法

```
TemplateClassName<arguments> objName(Constructor_parameters);
```

类模板的成员函数在类外定义

```
template<typename T>
ReturnType ClassName<T>::FuncName(parameters, ...) {
        // do_something
}
```

例如，上面的 `getElement` 函数在类外定义：

```
template<typename T>
T CArray<T>::getElement(int index) {
        return ptrElement[index];
}
```

类模板示例：

```
template <typename T1, typename T2>
class Pair {
public:
        T1 key;
        T2 value;
        Pair(T1 k, T2 v) : key(k), value(v) { }
        bool operator<(const Pair<T1, T2>& p) const;
};

template <typename T1, typename T2>
bool Pair<T1, T2>::operator<(const Pair<T1, T2>& p) const {
        return key < p.key;
}

int main() {
        Pair<string, int> student("Tom", 19); //实例化出一个类Pair<string, int>
        cout << student.key << " " << student.value;
        return 0;
}
```

也可以利用类模板实参推导(Class Template Argument Deduction，CTAD)机制完成实例化

```
template<typename T> class C {
    T mem;
public:
    C(T mem) : mem{mem} { }
};

int main() {
        C a(3); // 无需模板实参，推导出应生成 C<int> 类
}
```

函数模板作为类模板成员

类模板中的成员函数可以是一个函数模板，该成员函数只有在被调用时才会被示例化

```
template <typename T>
class A {
public:
        template <typename U>
        void func(U t) {
                cout << t;
        }
};

int main() {
        A<int> a;
        a.func('K');
        a.func('Hello');
        return 0;
}
```

- 为类模板中各类型参数指定了具体的数据类型后，即得到一个模板类
  - 编译器自动用模板实参替换类模板中的模板形参，生成模板类的代码
  - 如果模板实参不同，则得到的模板类是不同的，即同一个类模板的两个模板类是不兼容的

```
Pair<string, int>* p;
Pair<string, double> a;
p = &a; // error
```

类模板与派生

类模板从类模板派生

```
template <typename T1, typename T2>
class A {
        T1 v1;
        T2 v2;
};

template <typename T1, typename T2>
class B : public A<T2, T1> {
        T1 v3;
        T2 v4;
};

template <typename T>
```

```
class C : public B<T, T> {
        T v5;
};

int main() {
        B<int, double> obj1;
        C<int> obj2;
        return 0;
}
```

示例化后，`obj1`与`obj2`分别为以下结构：

`obj1`

```
class B<int, double> : public A<double, int> {
        int v3; double v4;
};
class A<double, int> {
        double v1; int v2;
};
```

`obj2`

```
class C<int> : public B<int, int> {
        int v5;
};
class B<int, int>:public A<int, int>{
   int v3; int v4;
};
class A<int, int> {
   int v1; int v2;
};
```

类模版从模板类派生

```
template <typename T1, typename T2>
class A {
        T1 v1;
        T2 v2;
};

template <typename T>
class B: public A<int, double> {
        T v;
};

int main() {
```

```
        B<char> obj1; //自动生成两个模板类: A<int, double>和B<char> return 0;
}
```

类模板从普通类派生

```
class A {
        int v1;
};

template <typename T>
class B: public A { // 所有从B实例化得到的类, 都以A为基类
        T v;
};

int main() {
        B<char> obj1;
        return 0;
        }
```

普通类从模板类派生

```
template <typename T>
class A {
        T v1;
        int n;
};

class B : public A<int> {
        double v;
};

int main() {
    B obj1;
        return 0;
}
```

非类型模板参数

模板参数不一定是类型，还可以是变量，称为非类型模板参数

- 可以用于替换模板中的变量

```
template <int N>
int multiply(int n) {
        return N * m;
}
```

```
int main() {
        std::cout << multiply<4>(3) << std::endl;
}
```

```
template <typename T, int N>
class Array {
private:
        T data[N] {};
public:
        void fill(const T& value) {
                for (int i = 0; i < N; ++i) {
                        data[i] = value;
                }
        }
};

int main() {
        Array<int, 5> arr;
        arr.fill(42);
}
```

由于非类型模板参数是在编译期间确定的，因此非类型模板参数必须是常量表达式

```
template <int N, int M>
int add() {
        return N + M;
}

int main() {
        add<1, 2>()                             // ok, 3
        int a = 2;
        add<1, a>();                    // error, a不是常量表达式
        constexpr int b = 2;    // 常量表达式
        add<1, b>();                    // ok, 3
        const int c = a;                // const不是常量表达式
        add<1, c>();                    // error, c不是常量表达式
}
```

Question

下面的两段程序有什么区别?

```
template<int N, int M>
int add() {
```

```
        return N + M;
}
add<1, 2>(); // 调用模板add，实例化为函数add<1, 2>
```

```
int add(int n, int m) {
    return n + m;
}
add(1, 2);   // 调用函数int add(int, int)
```

1. 模板 `add` 是有能力生成大量函数的语法
    - 当用不同的模板实参去实例化它时，会得到不同的函数
    - 从编译器角度来看，`add<1, 2>` 和 `add<3, 4>` 是两个不同的函数
2. 函数 add 永远只是那一个函数
    - 从编译器角度来看，`add(1, 2)` 和 `add(3, 4)` 调用的是同一个函数

补充：

标准模板库(STL)

C++标准

- 核心语言：C++语法
- 库：伴随编译器提供给程序员的一系列已编译好或尚未编译的代码，并在标准库头文件中对这些代码进行了声明
    - 标准模板库(Standard Template Library，STL)
        - C++创建时期，由Alexander Stepanov编写的一套基于模板的C++库，包含了一些常用数据结构和算法的模板的集合
        - 在 1994 年大部分被并入 C++ 标准，称之为“标准模板库”
        - 有了STL，不必再写大多的标准数据结构和算法，并且可获得非常高的性能

STL的设计理念

将程序需求抽象为STL：

- 将存储数据抽象为容器
    - 容纳各种数据类型的通用数据结构，是类模板
        - 可以容纳基本类型的变量，也可以容纳对象
        - 不同类型的容器具有不同的特点和效率，可根据具体需求来使用
- 将读写数据抽象为迭代器
    - 可用于依次存取容器中元素，类似于指针
        - 至少具备自增 `++`、解地址 `*` 操作
        - 普通的C++指针也是一种迭代器
- 将操作数据抽象为算法

    - 用于操作容器中的元素的函数模板
        - 算法本身与他们所操作的数据类型无关
        - 因此可以在从简单数组到高度复杂容器的任何数据结构上使用

容器

可以用于存放各种类型的数据(基本类型的变量， 对象等)的数据结构，都是类模板

容器分为三种：

- 顺序容器(又称序列容器)
    - `vector`，`deque`，`list`
- 关联容器
    - 有序关联容器: `set`，`multiset`，`map`，`multimap`
    - 无序关联容器: `unordered_set`，`unordered_map`，`unordered_multiset`，`unordered_multimap`
- 容器适配器
    - `stack`，`queue`，`priority_queue`

对象被插入容器中时，被插入的是对象的一个 复制品

顺序容器

容器内的元素并非排序的，元素的插入位置同元素的值无关，元素的顺序可以被控制

- `vector`：头文件`<vector>`
    - 动态数组。元素在内存连续存放。随机存取任何元素都能在常数时间完成。在尾端增删元素具有较佳的性能(大部分情况下是常数时间)
- `deque`：头文件`<deque>`
    - 双向队列(也是一个动态数组)。元素在内存连续存放。随机存取任何元素都能在常数时间完成(但次于vector)。在两端增删元素具有较佳的性能(大部分情况下是常数时间)
- `list`：头文件`<list>`
    - 双向链表。元素在内存不连续存放。在任何位置增删元素都能在常数时间完成。不支持随机存取。

容器的成员函数

大部分标准库容器共有的成员函数

- `empty`：判断容器中是否有元素
- `max_size`：容器中最多能装多少元素
- `size`：容器中元素个数
- `swap`：交换两个容器的内容

- 相当于按词典顺序比较两个容器的运算符:
  - =, < , <= , > , >=, == , !=

顺序容器的常用成员函数

- `front`：返回容器中第一个元素的引用
- `back`：返回容器中最后一个元素的引用
- `push_bac`k: 在容器末尾增加新元素
- `pop_back`：删除容器末尾的元素
- `clear`：从容器中删除所有元素

例：`vector` 容器

可以像使用数组一样使用 `vector` 容器

```
#include <iostream>
#include <vector> //vector定义于<vector>头文件
using namespace std;

int main() {
        vector<int> a{1, 2, 3}; //定义vector对象
        vector b{4, 5}; //可省略类型实参

        //可像数组一样使用vector
        cout << a[0] << endl; // 输出 1
        cout << b[1] << endl; // 输出 4
}
```

在 `vector` 容器的尾部添加一个元素

```
#include <iostream>
#include <vector>
using namespace std;

// 遍历输出一个vector的全部元素
void print(const vector<int>& v) {
        for (int i = 0; i < v.size(); ++i) {
                cout << v[i] << " ";
        }
        cout << endl;
}

int main() {
        vector a{1, 2, 3};
        print(a); // 输出 1 2 3
        a.push_back(4);
```

```
        print(a); // 输出 1 2 3 4
}
```

迭代器

用于指向顺序容器和关联容器中的元素

- 迭代器用法和指针类似
- 有const 和非const 两种，通过迭代器可以读取它指向的元素，通过非const迭代器还能修改其指向的元素

定义一个容器类的迭代器的方法

- 非 `const` 迭代器
  - `容器类名::iterator 迭代器名;`
- `const` 迭代器
  - `容器类名::const_iterator 迭代器名;`

访问一个迭代器指向的元素：

- `*it`： 返回迭代器it指向的元素
- 迭代器可以执行 `++` 操作，指向容器中的下一个元素
  - 如果迭代器到达了容器中的最后一个元素的后面，此时再使用它，就会出错，类似于使用nullptr或未初始化的指针一样
- 迭代器有多种类型
  - 不同容器上支持的迭代器功能强弱有所不同
  - 容器的迭代器的功能强弱，决定了该容器是否支持STL中 的某种算法

容器中有关迭代器的成员函数

顺序容器和关联容器中都有的成员函数：

- `begin`：返回指向容器中第一个元素的迭代器
- `end`：返回指向容器中最后一个元素后面的位置的迭代器
- `rbegin`：返回指向容器中最后一个元素的迭代器
- `rend`：返回指向容器中第一个元素前面的位置的迭代器
- `erase`：删除迭代器指向的元素(可能会使该迭代器失效)，或删除一个区间，返回被删除元素后面的那个元素的迭代器

例： `vector` 容器的迭代器

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
```

```
        vector<int> v; // 一个存放int元素的vector容器对象，一开始里面没有元素
        v.push_back(1);
        v.push_back(2);
        v.push_back(3);
        v.push_back(4);

        vector<int>::const_iterator i; // 常量迭代器
        for(i = v.begin(); i != v.end(); i++) {
                cout << *i << ", ";
        }
        cout << endl;

        vector<int>::iterator j; // 非常量迭代器
        for(j = v.begin(); j != v.end(); j++) {
                *j *= 100;
        }

        for( i= v.begin(); i!= v.end(); i++ ) {
                cout << *I << ", ";
        }
        cout << endl;

        return 0;
}
```

算法

STL中提供能在各种容器中通用的算法，比如插入/删除/查找/排序等，大约有70种标准算法

- 大多数在 `<algorithm>` 头文件中定义
- 算法就是一个个函数模板
- 算法通过迭代器来操纵容器中的元素
- 许多算法需要两个参数，一个是起始元素的迭代器，一个是终止元素的后面一个元素的迭代器
    - 排序和查找
- 有的算法返回一个迭代器
    - 如 `find()` 算法，在容器中查找一个元素，并返回一个指向该元素的迭代器
- 算法可以处理容器，也可以处理C语言的数组

例：`find()` 函数

其函数签名为：

```
template <class InputIterator, class T>
InputIterator find (InputIterator first, InputIterator last, const T& val);
```

- `first`和`last`这两个参数都是容器的迭代器，它们给出了容器中的查找区间起点和终点
  - 这个区间是个左闭右开的区间
- val参数是要查找的元素的值
- 函数返回值是一个迭代器
  - 如果找到，则该迭代器指向被找到的元素
  - 如果找不到，则该迭代器指向查找区间终点

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
        int array[10] = {10, 20, 30, 40};
        vector<int> v;
        v.push_back(1);
        v.push_back(2);
        v.push_back(3);
        v.push_back(4);
        vector<int>::iterator p;
        p = find(v.begin(), v.end(), 3); //在整个容器中查找3
        if (p != v.end()){
                cout << *p << endl;
        }
        p = find(v.begin(), v.end(), 9); //在整个容器中查找9
        if (p == v.end()){
                cout << "not found" << endl;
        }
        p = find(v.begin()+1, v.end()-1, 1); //在区间[2, 4)查找1
        if (p != v.end()){
                cout << *p << endl;
        }

        int *pp = find(array, array+4, 20);
        cout << *pp << endl;

        return 0;
}
```

补充：模板实参推导

当未指定（或部分指定）模板实参列表时，通过推导确定模板实参

- 用T表示，可以推导什么类型？

```
template <typename T>
void f(T x);
```

```
void g(int, double);

int main() {
        int a = 1;
        f(2.3);         // T = double
        f(true);        // T = bool
        f('3');         // T = char
        f(&a);          // T = int*
        f(g);           // T = void(*)(int, double)
}
```

- 一般不进行隐式类型转换
- 不推导引用类型和cv限定(const)
- 忽略cv限定

Hint

"cv限定"是C++中的一个术语，"cv"是 `const` 和 `volatile` 的缩写。这两个关键字可以用来修饰变量，限定其属性
`const`： 一个 `const` 变量一旦被赋值，就不能再改变。这可以用来创建不可变的变量或对象
`volatile`： 一个 `volatile` 变量表明它可能在外部被改变，这意味着编译器在优化代码时不能假设这个变量的值不会改变
当我们说"不推导引用类型和cv限定"，意思是在模板参数推导过程中，编译器不会推导出引用类型或者带有 `const` / `volatile` 限定的类型。例如，如果传递一个 `const int` 给一个模板函数，推导出的类型将是 `int`，而不是 `const int`

```
int &r = a;
f(r);   // T = int
const int b = 1;
f(b);   // T = int
int p[3] {1, 2, 3}
f(p);   // T = int*
```

如果对 `T` 进行了 `*`、`&`、`const` 等操作，则按照解析类型的方式进行推导

```
template <typename T>
void f(T& x);
int main() {
        double b = 2.0;
        f(b);   // T = double
        f(2);   // error，2是右值，不能绑定到左值引用
}

部分指定模板实参时，推导缺省的部分（类似函数参数缺省）
```

```cpp
template <typename T1, typename T2>
void f(T1 x, T2 y);
int main() {
        f(1, 2.0);              // T1 = int, T2 = double
        f<int>(1, 2.0); // T1 = int, T2 = double
        f(1.0, 2);              // T1 = double, T2 = int
}
```

- 如果推导矛盾，则编译错误

类模板实参推导

- 与函数模板实参推导类似，不过是利用构造函数/构造函数模板进行推导
- 不能进行部分推导

补充：重载决议

函数调用时，编译器：

1. 名字查找；
2. 可能涉及模板实参推导；
3. 如果找到多个名字，进行重载决议
    - 重载决议将尝试挑选唯一匹配的实体

- 原则：越自然越好

1. 确定可行函数
2. 从可行函数里尝试挑选唯一最佳可行函数

- 具体规则复杂，大致思想：

1. 优先匹配无需隐式类型转换的函数
2. 如果都需要隐式类型转换，找尽可能少得进行“不合理”的隐式类型转换
3. 如果2相同，优先找函数而非函数模板特化
4. 如果都是函数模板特化，优先选更“特殊”的模板特化（简单地说，接受更少的类型）

- 如果能确定唯一最佳函数，则调用；否则报错

```cpp
template <typename T>
T max(T a, T b) {
        cout << "TemplateMax" << endl;
        return 0;
}
```

```
double max(double a, double b) {
        cout << "MyMax" << endl;
        return 0;
}

int main() {
        int i = 4, j = 5;
        max(1.2, 3.4);          // MyMax
        max(i, j);                      // TemplateMax
        max(1.2, 3);            // MyMax
        return 0;
}
```

Note

模板、函数、类综合

- 类：普通类、模板类
- 函数：普通函数、函数模板
- 可以任意组合：
- 普通类含有普通函数成员
- 普通类含有函数模板成员
- 模板类含有普通函数成员
- 模板类含有模板函数成员

补充：模板形参包

- 形参包的声明形式

```
template <typename... TsName>
FuncType FuncName(TsName... argsName)
```

- 可以接受任意数量的实参，增强可扩展性
- 可以用省略号进行包展开
- 包展开只能在特定的语境下使用：
  - 函数/模板实参列表、部分初始化器……

例如：

```
template <typename... Types>
void print(Types... args) {
        std::vector v{args...};
        for (auto& i : v) {
                std::cout << i << std::endl;
```

```
        }
}
```

逆天玩法

包展开

对于一个函数形参包 `args`，它更常见的使用方式是**包展开**。包展开最简单的做法，就是在后面加上 `...`，这样 `args...` 就会被替换为每一个形参的具体值用逗号分隔开来的形式。

比如下面的代码会从形参包生成包含全部形参作为元素的 `std::vector`：

```
template <typename... Ts>
std::vector<int> f(Ts... args) {
    return { args... };
}
f(1, 2, 3); // 返回包含 {1, 2, 3} 的 std::vector
```

任何期望“逗号分隔的值的列表”的地方都可以出现形参包的展开，比如函数调用表达式的实参列表。下面的代码可以将形参包的首个形参的值输出：

```
template <typename T, typename... Us>
void outputFirst(T t, Us...) {
    std::cout << t << std::endl;
}
template <typename... Ts>
void f(Ts... args) {
    outputFirst(args...); // 输出 args 中首个形参
}
```

从而，我们可以通过适当的“递归”和重载，实现处理任意多参数的函数

例如：我们将实现一个print函数，将任意多的形参通过 `std::cout` 输出

```
#include <iostream>
#include <string>
template <typename T>
void print(const T& x) {
    std::cout << x << std::endl;
}

template <typename T, typename... Args>
void print(const T& x, const Args&... args) {
    std::cout << x; // 输出第一个
    print(args...); // “递归”
}
```

```
void print() {
    std::cout << std::endl;
}
int main() {
    std::string n("nullptr");
    print(false);
    print();
    print("hello", " world");
    print("do not ", "dereference ", n);
    print(1, '+', 1LL, "=", 2.0);
}
```

展开模式

对于函数形参包 `args`，`args...` 只是最简单的一种包展开形式。更通用的形式是**展开模式**。所谓的**模式**，就是指包含形参包的表达式如 `args + 1`；将模式展开，就会将形参包中的每个形参，按照模式的形式展开。说起来太抽象，来看例子：

```
template<typename... Ts>
void foo(Ts... args) {
    bar( (args + 1)... );
}
foo(1, 2, 3); // 形参包 args 的值为 1, 2, 3
```

这里的 `(args + 1)` 就是模式，后面的省略号对这个模式展开；展开的结果就如同 `args_0 + 1, args_1 + 1, args_2 + 1`，也就是将每个形参都以 `+ 1` 的形式写出。因此这里传递给函数 `bar` 的实参，就是 `2, 3, 4`。

类似地，我们也可以写出函数 `makeSqrtVec()` 输出任意多形参各自的平方根：

```
#include <iostream>
#include <vector>
#include <cmath>
template <typename... Ts>
std::vector<double> makeSqrtVec(Ts... args) {
    return std::vector<double>{(sqrt(args))...}; // 展开模式
}
int main() {
    for (auto i : makeSqrtVec(1, 4, 9, 16, 25)) {
        std::cout << i << " ";
    }
    std::cout << std::endl;

    for (auto i : makeSqrtVec(8, 7, 6)) {
        std::cout << i << " ";
    }
    std::cout << std::endl;
```

```
    for (auto i : makeSqrtVec()) {
        std::cout << i << " ";
    }
}
```

折叠表达式

折叠表达式允许在参数包中连续“折叠”地做二元运算，并得到一个最终的结果。比如对于形参包 `1, 2, 3, 4`，在其上左折叠地做“加法”，效果就如同 `(((1+2)+3)+4)`。类似地还有右折叠，只不过是先对最后两个形参做运算。

左折叠的折叠表达式写成 `(... op args)`，其中 `op` 是二元运算符，`args` 是形参包，如 `(... + args)` 就是左折叠地做加法。右折叠就是将省略号放在运算符右侧：`(args op ...)`。注意折叠表达式的外围括号是必需的。

折叠表达式既可以作用在函数形参包上，也可以作用在模板形参包上。比如对于 `template <int... Ns> /* ... */` 这种模板，出现 `(... * Ns)` 的场合就会被替换为所有 `Ns` 非类型实参的乘积。

注：很显然这种折叠要求形参包的形参个数必须大于等于 2。对于数量更少的形参包，可以用带初始值的折叠表达式，写成 `(init op ... op args)` 或 `(args op ... op init)`

例如：下面我将实现函数模板 `sum` 返回所有实参的和、函数模板 `sumOfSquare` 返回所有实参的平方和、类模板 `Sum` 具有静态常量成员 `value` 以计算模板实参的和。

```
#include <iostream>
#include <string>
template <typename... Ts>
auto sum(Ts... args) {
    return (... + args);
}

template <int... Ts>
struct Sum {
    static constexpr int value = (... + Ts);
};

template <typename... Ts>
auto sumOfSquare(Ts... args) {
    return sum((args * args)...);
}
int main() {
    std::cout << sum(6, 6, 6) << std::endl;
    std::cout << Sum<1, 2, 3, 4, 5>::value << std::endl;
    std::cout << sum(1, 4, 2, 8, 5, 7) << std::endl;
    std::cout << Sum<4, 3, 9, 9>::value << std::endl;
    std::cout << sum(std::string{}, "Hello", ", ", "world", '!') << std::endl;
```

```
    std::cout << sumOfSquare(3, 4, 12, 13) << std::endl;
}
```

运算符可以折叠，同时，逗号也可以折叠，所以我们可以使用一种更tricky的方式来解决上面的问题软件设计实践 > Lecture08-C++泛型程序设计(1) 模板与STL初步 > 包展开

```
#include <iostream>
#include <string>
template <typename... Ts>
void print(const Ts&... args) {
    ((std::cout << args), ...); // 逗号包展开
    std::cout << std::endl;
}

void print() {
    std::cout << std::endl;
}
int main() {
    std::string n("nullptr");
    print(false);
    print();
    print("hello", " world");
    print("do not ", "dereference ", n);
    print(1, '+', 1LL, "=", 2.0);
}
```

Note

- 可复用性
  - 使用模板可以针对不同数据类型复用函数和类的实现 - 使用STL可以复用高效的数据结构和算法

Lecture09-C++泛型程序设计(2) STL基础

顺序容器

回顾：顺序容器的特点

容器内的元素并非排序的，元素的插入位置同元素的值无关，元素的顺序可以被控制

- `vector`:头文件`<vector>`
  - 动态数组。随机存取任何元素都能在常数时间完成。在尾端增删元素具有较佳的性能(大部分情况下是常数时间)
- `deque`:头文件`<deque>`

    - 双向队列(也是一个动态数组)。随机存取任何元素都能在常数时间完成(但次于vector)。在两端增删元素具有较佳的性能 (大部分情况下是常数时间)
- `list`:头文件`<list>`
    - 双向链表。在任何位置增删元素都能在常数时间完成。不支持随机存取。

常用函数:

- `front`：返回容器中第一个元素的引用
- `back`：返回容器中最后一个元素的引用
- `push_back`：在容器末尾增加新元素
- `pop_back`：删除容器末尾的元素

`std::vector`

- `vector`代表一个长度可变化的数组

```
#include <iostream>
#include <vector>

void print(const std::vector<int>& v) {
        for (size_t i = 0; i < v.size(); ++i) {
                std::cout << v[i] << " ";
        }
        std::cout << std::endl;
}

int main() {
        vector a{1, 2, 3};
        print(a); // 输出 1 2 3
        a.push_back(4);
        print(a); // 输出 1 2 3 4
}
```

- 用`vector`实现二维数组

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
        vector<vector<int>> v(3); //v有3个元素，每个元素都是vector<int>容器
        for(int i = 0; i < v.size(); ++i)
                for(int j = 0; j < 4; ++j)
                        v[i].push_back(j);

        for(int i = 0; i < v.size(); ++i) {
                for(int j = 0; j < v[i].size(); ++j)
```

```
                        cout << v[i][j] << " ";
                cout << endl;
        }
        return 0;
}
```

`std::deque`

与 `std::vector` 很像，也是一个长度可变的数组

- 比 `std::vector` 多提供了两个成员函数 `push_front()` 和 `pop_front()`，即从头部插入或删除元素
- `std::deque` 的元素访问(即 `operator[]`)稍慢于 `std::vector`

```
#include <iostream>
#include <deque>
using namespace std;

int main() {
        deque<int> d;
        d.push_back(1);
        d.push_back(2);
        d.push_back(3);
        d.push_front(0);
        d.push_front(-1);
        d.push_front(-2);
        for(int i = 0; i < d.size(); ++i) {
                cout << d[i] << " ";
        }
        cout << endl;
        // 输出 -2 -1 0 1 2 3
        d.pop_back();   // 现在d中的元素是-2 -1 0 1 2
        d.pop_front();  // 现在d中的元素是-1 0 1 2

        return 0;
}
```

`std::list`

`std::list` 是一个双向链表

- 在任何位置插入删除都是常数时间
- 不支持随机存取(即不能使用 `[]` 运算符)
- 除了具有所有顺序容器都有的成员函数以外，还支持8个成员函数
  - `push_front` : 在头部插入
  - `pop_front` : 删除头部的元素

- `sort`：排序(`list`不支持STL的算法`sort`)
- `remove`：删除和指定值相等的所有元素
- `unique`：删除所有和前一个元素相同的元素
- `merge`：合并两个链表，并清空被合并的那个
- `reverse`：颠倒链表
- `splice`：在指定位置前面插入另一链表中的一个或多个元素，并在另一链表中删除被插入的元素

迭代器

基于范围的for循环

C++11引入了基于范围的for循环，可以方便地遍历容器中的元素

```
#include <iostream>

int main() {
        int a[] = {1, 2, 3, 4, 5};
        for (int i : a) {
                std::cout << i << " ";
        }
        std::cout << std::endl;
        return 0;
}
```

这种基于范围的for循环可以作用在C风格数组上，也可以作用在STL容器上，这是如何实现的？

我们可以抽象成以下过程：

1. 取出某个指向迭代器头部的“指针”`begin`
2. 取出指针`begin`对应的数据
3. 执行循环体
4. 将指针`begin`向后移动一个位置
5. 重复2-4，直到指针`begin`指向迭代器尾部

伪代码：

```
auto begin = begin of iterator;
while (begin != end of iterator) {
        auto i = getVal(begin);
        // do something
        begin = next(begin);
}
```

迭代器

迭代器是STL提供的某种类似指针的数据结构

- `it` 是某个数据类型的变量，只要 `++it` 和 `*it` 是合法表达式， `it` 就是迭代器
- 通过 `*it` 可以访问它“指向”的元素
- 通过 `++it` 可以移动到“下一个”元素
- 裸指针就是迭代器
- 定义一个容器类的迭代器

```
容器类名::iterator 迭代器变量名
容器类名::const_iterator 迭代器变量名
```

- 容器提供了 `begin()` 和 `end()` 函数来确定界限
  - `begin()` 返回指向容器中第一个元素的迭代器
  - `end()` 返回指向容器中最后一个元素后面的位置的迭代器

迭代器的种类

STL中的迭代器按功能由弱到强分为6种

1. 输入迭代器:Input iterators提供对数据的只读访问
2. 输出迭代器:Output iterators提供对数据的只写访问
3. 前向迭代器:Forward iterators提供读写操作，并能一次一个地向前推进迭代器
4. 双向迭代器:Bidirectional iterators提供读写操作，并能一次一个地向前和向后移动
5. 随机访问迭代器:Random access iterators提供读写操作，并能在数据中随机移动
6. 连续迭代器:Contiguous iterator提供读写操作，并能在数据中随机移动，相邻的容器元素在内存中物理相邻

- 编号大的迭代器拥有编号小的迭代器的所有功能，能当作编号小的迭代器使用
- 不同迭代器所能进行的操作
  - 所有迭代器: `++p` ， `p++`
  - 输入迭代器: 通过 `*p` 读取数据
  - 输出迭代器: 通过 `*p` 修改数据
  - 前向迭代器: 以上全部，以及 `p==q` ， `p!=q`
  - 双向迭代器: 以上全部，以及 `--p` ， `p--`
  - 随机访问迭代器:以上全部，以及
    - 移动i个单元: `p+= i` , `p -= i` , `p + i` , `p - i`
    - 大于/小于比较: `p < q` , `p <= q` , `p > q` , `p>= q`
    - 下标 `p[i]` : p后面的第i个元素的引用
  - 连续迭代器: 以上全部，以及 `p-q` 恰好是其所“指向的元素”的地址的差

不同容器所支持的迭代器种类不同：

容器	支持的迭代器
数组	连续
`vector`	连续
`deque`	随机访问
`list`	双向
`set` / `multiset`	双向
`map` / `multimap`	双向
`forward_list`	前向
`stack`	不支持
`queue`	不支持
`priority_queue`	不支持

遍历 `vector`

```
int main() {
        vector<int> v(100); //包含100个元素的vector
        int i;
        for (i = 0;i < v.size(); i ++)
                cout << v[i]; //根据下标随机访问

        vector<int>::const_iterator ii;
        for (ii = v.begin(); ii != v.end(); ++ii)
                cout << *ii;
        for (ii = v.begin(); ii < v.end(); ++ii)
                cout << *ii;

        //间隔一个输出
        ii = v.begin();
        while (ii < v.end()) {
                cout << *ii;
                ii = ii + 2;
        }
}
```

遍历 `list`

- 正确做法

```
list<int> v;
list<int>::const_iterator ii;
```

```
for (ii = v.begin(); ii != v.end(); ++ii)
        cout << *ii;
```

- 错误做法

```
//双向迭代器不支持<
for (ii = v.begin(); ii < v.end(); ++ii)
        cout << *ii;
//list不支持[]运算符
for (int i = 0; i < v.size(); i++)
        cout << v[i];
```

基于迭代器的容器成员函数

- 增加元素
  - 如果 `pos` 指向了第n个元素，那么调用 `insert` 成员函数之后，就会将 `value` 这个值插入在第n个元素之前

```
iterator insert(const_iterator pos, const T& value);
```

- 删除元素
  - 如果 `pos` 指向了第n个元素，那么调用 `erase` 成员函数之后，就会删除第n个元素

```
iterator erase(const_iterator pos);
```

```
vector a{2, 3};
// 在首个元素2之前插入1
a.insert(a.begin(), 1);
// 现在a是{1, 2, 3}
// 在中间插入
auto secondIt = a.begin() + 2;
a.insert(secondIt, 5);
// 现在a是{1, 2, 5, 3}
a.erase(a.begin()); //删除第一个元素
//现在a是{2, 5, 3}
```

⚠ Caution

注意：

- 当插入或删除元素时，整个容器的构造可能发生改变; 此时，原先的迭代器可能不再适用，变成了类似“野指针”一样的“野迭代器”

- 不要在基于范围的for循环中增删元素，因为其中用到的迭代器是循环之前得到的

STL算法

我们可以基于迭代器实现“通用”的算法

- 通用是指算法可以适用于不同类型的容器
  - 通过迭代器的类型来约定可适用的容器
- 例:通用的冒泡排序
  - 只要一个容器上的迭代器满足前向迭代，就可以使用冒泡排序算法

```
template <typename Iter>
void bubbleSort(Iter begin, Iter end) {
        for (Iter i = begin; i != end; ++i) {
                for (Iter j = begin; j != i; ++j) {
                        if (*i < *j) {
                                std::swap(*i, *j);
                        }
                }
        }
}
```

STL中提供能在各种容器中通用的算法，比如插入/删除/查找/排序等，大约有70种标准算法

- 大多数定义在 `<algorithm>` 和 `<numeric>` 头文件中
  - 掌握查阅CppReference使用STL算法的能力
- 算法就是一个个函数模板
- 算法通过迭代器来操纵容器中的元素
- 许多算法需要两个参数，一个是起始元素的迭代器，一个是终止元素的后面一个元素的迭代器
  - C++20提出“范围”概念来表示这种具有起始和终止元素迭代器的对象，从而简化STL算法的调用
- 有的算法返回一个迭代器
  - 如 `find()` 算法，在容器中查找一个元素，并返回一个指向该元素的迭代器
- 算法可以处理容器，也可以处理C语言的数组

算法示例

`std::copy`

函数签名：

```
template <class InIt, class OutIt>
OutIt copy (InIt first, InIt last, OutIt result);
```

该函数对在区间$[0, last - first)$中的每个N执行一次 `*(result + N) = *(first + N)`，返回x+N

函数实现：

```
template <class InIt, class OutIt>
OutIt copy (InIt _F, InIt _L, OutIt _X) {
        for (; _F != _L; ++_X, ++_F) {
                *_X = *_F;
        }
        return _X;
}
```

应用：

```
int main() {
        vector a{1, 3, 5, 7, 9};
        vector<int> b(5); // 持有 5 个零的 vector

        std::copy(a.begin(), a.end(), b.begin());

    for (int i : b) {
        cout << i << endl;
        } // 输出 1 3 5 7 9
}
```

`std::back_inserter()` 函数可以用于向顺序容器的尾部插入元素

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <iterator>

int main() {
        std::vector<int> a {1, 3, 5, 7, 9};
        std::vector<int> b;
        std::copy(a.begin(), a.end(), std::back_inserter(b));

        for (int i : b) {
                std::cout << i << " ";
        } // 输出 1 3 5 7 9
}
```

`std::sort`

函数签名：

```
template <typename RanIt>
void sort (RanIt first, RanIt last);
```

该函数将 `first` 和 `last` 迭代器指向的区间进行排序

- 判断x是否应比y靠前，就看x < y是否为 `true`

应用：

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
        vector a{4, 1, 6, 2};
        sort(a.begin(), a.end());
        for (auto i : a) {
        cout << i << ' ';
    } // 输出 1 2 4 6
}
```

Question

如何按降序排序?

重载为：

```
template <typename RanIt, typename Pred>
void sort (RanIt first, RanIt last, Pred pr);
```

将first和last迭代器指向的容器范围按升序排序

- 判断x是否应比y靠前，就看 `pr(x, y)` 是否为 `true`

Note

`sort()` 基于元素的交换排序，如果 `pr(a, b)` 为true，则将a排到b前面。当不停地交换到任意两个元素a和b都符合 `pr(a, b)` 为 `true` 时，整个序列就排好了

应用：

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

int main() {
        vector a{4, 1, 6, 2};
        sort(a.begin(), a.end(), greater<int>());
        for (auto i : a) {
        cout << i << ' ';
    } // 输出 6 4 2 1
}
```

Question

`greater<int>()` 是什么?

`greater` 是一个重载了 `()` 运算符的类模板

```
template <typename T>
struct greater {
        bool operator()(const T& x, const T& y) const {
                return x > y;
        }
};
```

此处，`greater<int>()` 是一个"函数对象"(见下文软件设计实践 > 函数对象)

一般来说，如果这个容器/数组里的对象定义了比较运算符，那么它就可以传入到sort里排序，并且可以用pr参数控制升序或者降序

- 没有定义比较运算符怎么办?
- 可以传入一个函数对象，这个函数对象的 `()` 运算符重载了比较运算符

```
struct Coor {
        int x, y;
};

int main() {
        Coor a[] {{1, 2}, {3, 4}, {5, 6}};

        sort(a, a + 3) // error， Coor没有定义比较运算符

        // Method#1
```

```
        sort(a, a + 3, [](const Coor& x, const Coor& y) {
                return x.x < y.x;
        });

        // Method#2
        bool cmp(const Coor& x, const Coor& y) {
                return x.x < y.x;
        }
        sort(a, a + 3, cmp);

        // Method#3
        struct CoorCmp {
                bool operator()(const Coor& x, const Coor& y) {
                        return x.x < y.x;
                }
        };
        sort(a, a + 3, CoorCmp());
}
```

`std::list` 中的 `sort` 成员函数

`std::list` 容器的迭代器不支持随机访问，所以不能用标准库中 `sort` 函数对它进行排序

- `std::list` 自己的 `sort` 成员函数

```
list<T> lst
lst.sort(compare);      //compare函数可以自己定义
lst.sort();                     //无参数版本，按<排序
```

`std::copy_if`

在复制之前会检查当前被复制的元素是否满足条件，只有满足时才会复制

- 其中，最后一个参数是可调用对象，如果其作用在当前元素上返回 true ，那么该元素就会被复制

应用：

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
using namespace std;
int main() {
        std::vector<int> a{4, 1, 6, 2, 9, 5};
        std::vector<int> b;
        copy_if(a.begin(), a.end(), back_inserter(b), [](int x) {
```

```
                return x < 5;
        });     // 这样的写法是不是很啰嗦? C++20给出了新写法
    for (auto i : b) {
        std::cout << i << ' ';
        } // 输出 4 1 2
}
```

`std::transform`

函数签名:

```
template< typename InIt, typename OutIt, typename Unop>
OutIt transform(InIt first, InIt last, OutIt x, Unop uop);
```

对$[first, last)$中的每个迭代器`i`,执行`uop(*i)`;并将结果依次放入从`x`开始的地方

- 要求`uop(*i)`不得改变`*i`的值。

应用:

将`a`中的每个元素平方后放入`b`中

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <iterator>
using namespace std;

int main() {
        std::vector<int> a{1, 2, 3, 4, 5};
        std::vector<int> b;

        transform(a.begin(), a.end(), back_inserter(b), [](int x) {
                return x * x;
        });
        for (auto i : b) {
                cout << i << ' ';
        } // 输出 1 4 9 16 25
}
```

从标准输入读入一组整数，筛选出其中的正数，取前5个，然后对这5个数分别取平方根，最后输出这5个数

```
int main() {
        vector<int> a;
        // 输入
        int x;
```

```
        while (cin >> x) {
        a.push_back(x);
    }
        // 筛选出正数
        vector<int> b;
        copy_if(a.begin(), a.end(), back_inserter(b), [](int x) {
                        return x > 0;
        });
        // 取前 5 个
        if (b.size() > 5) {
                b.resize(5);
        }
        // 取平方根
        vector<double> c;
        transform(b.begin(), b.end(), back_inserter(c), [](int x) {
                return sqrt(x);
        });
        // 输出
        for (auto i : c) {
                cout << i << ' ';
        }
}
```

`std::accumulate`

函数签名：

```
template<typename InIt, typename T, typename Pred>
T accumulate(InIt first, InIt last, T val, Pred pr);
```

对$[first, last)$中的每个迭代器`i`，执行`val = pr(val, *i)`，返回最终的`val`

应用：

下面代码的功能是：计算`a`中所有元素的和

```
#include <iostream>
#include <vector>
#include <numeric> // accmulmate不定义在<algorithm>
using namespace std;

int main() {
    vector a{1, 2, 3, 4, 5, 6};
        // 初始值是 0，二元函数是 (a, b) -> a + b
        auto sum = accumulate(a.begin(), a.end(), 0, plus<int>());
    cout << sum << endl;
}
```

```
int sumSquares(int total, int value) { return total + value * value; }
template<typename T>
class SumSquaresClass {
public:
    const T operator() (const T& total,const T& value) {
                return total + value * value;
        }
};

template<typename T>
class SumPowers {
private:
        int power;
public:
    SumPowers(int p): power(p) { }
    const T operator() (const T& total, const T& value) {
                //计算value的power次方, 加到total上
                T v = value;
                for(int i = 0; i < power - 1; ++i)      {
                        v = v * value;
                }
                return total + v;
        }
};

int main() {
        int a[5] = {1, 2, 3, 4, 5};
        vector<int> v(a);
        int result = accumulate(v.begin(), v.end(), 0, sumSquares);
        cout << "平方和:" << result << endl;

        SumSquaresClass<int> s;
        result = accumulate(v.begin(), v.end(), 0, s);
        cout << "平方和:" << result << endl;
        result = accumulate(v.begin(), v.end(), 0, SumPowers<int>(3));
        cout << "立方和:" << result << endl;
        result = accumulate(v.begin(), v.end(), 0, SumPowers<int>(4));
        cout << "4次方和:" << result;
        return 0;
}
```

函数对象

- 重载了 operator() 的类类型对象
- 简单来说就是，可以像函数一样使用的对象

函数对象和函数的区别:

- 函数对象可以携带局部的信息
- 原生函数只能访问函数参数或者全局的信息

补充：std::function

如果我要写一个函数，可以同时接受函数指针、Lambda表达式、函数对象作为参数

- 方法一:可以使用模板
- 方法二:使用 std::function

STL的 `<functional>` 里提供了常用的函数对象类模板

- 算数类型
  - `plus<T>` : 加法
  - `minus<T>` : 减法
  - `multiplies<T>` : 乘法
  - `divides<T>` : 除法
  - `modulus<T>` : 取模
  - `negate<T>` : 取反
- 关系类型
  - `equal_to<T>` : 等于
  - `not_equal_to<T>` : 不等于
  - `greater<T>` : 大于
  - `greater_equal<T>` : 大于等于
  - `less<T>` : 小于
  - `less_equal<T>` : 小于等于
- 逻辑类型
  - `logical_and<T>` : 逻辑与
  - `logical_or<T>` : 逻辑或
  - `logical_not<T>` : 逻辑非

可调用对象

可以出现在函数调用运算符 `()` 左侧的对象，称为可调用对象，常见的可调用对象包括:

- 函数
- 函数指针
- 可转换到函数指针的对象
- 重载了 `operator()` 的对象(即“函数对象”)
- Lambda表达式
  - 实际上是一个重载了 `()` 运算符的匿名类的对象
  - 捕获列表为空的Lambda表达式可隐式转换为函数指针类型

- 为了STL算法的可复用性，许多算法的“子操作”应该是参数化的，需要以可调用对象作为参数
  - 根据函数对象参数个数，STL算法用到的函数对象主要包括三种:
    - 没有参数(如 `generate` 算法)
    - 一个参数(如 `copy_if` 算法)
    - 两个参数(如 `sort` 算法)

算法的种类

STL的算法大致可以划分为七类对象，适用于所有容器，它们的时间复杂度都是$O(n)$

- 变值算法: 此类算法会修改源区间或目标区间元素的值。值被修改的那个区间，不可以是属于关联容器的
- 删除算法: 删除一个容器里的某些元素
- 变序算法: 改变容器中元素的顺序，但是不改变元素的值。不适用于关联容器。此类算法复杂度都是$O(n)$
- 排序算法: 算法复杂度一般是$O(n \times \log(n))$。需要随机访问迭代器的支持，因而不适用于关联容器和list
- 有序区间算法: 要求所操作的区间是已经从小到大排好序的，而且需要随机访问迭代器的支持
- 数值算法: 进行各类数值运算
- 大多重载的算法都有两个版本
  - 其中一个是用 `==` 判断元素是否相等，或用 `<` 来比较大小
  - 另一个版本多出来一个类型参数Pred，以及函数形参 Pred op，该版本通过表达式op(x, y)的返回值是ture还是 false，来判断x是否等于y，或者x是否小于y

补充：C++20中的范围概念

我们在使用STL算法时，总是成对地使用迭代器：

- `std::sort(a.begin(), a.end());`
- `std::find(a.begin(), a.end(), v);`
- `std::copy(a.begin(), a.end(), b.begin());`
- ...

它们明确地指代“a”这个范围

那么，什么是范围?

- 范围是能指示一系列元素的一对迭代器。
  - 在不引起歧义的情况下，称首迭代器为“迭代器”，尾迭代器为“哨位”。
- 容器的 begin、end 成员返回一对指示范围的迭代器
- 称这样的对象满足范围概念

- 数组也满足范围概念(虽然没有 `begin` 、 `end` 成员，但可以使用 `std::begin` 和 `std::end`)

我们可以通过将 `std::ranges::begin` 和 `std::ranges::end` 作用到一个对象上，获得它的范围迭代器和哨位

```
#include <algorithm>
#include <vector>
int main() {
    std::vector<int> a{2, 1, 3, 4, 0};
    int b[5]{2, 1, 3, 4, 0};
    std::sort(std::ranges::begin(a), std::ranges::end(a));
    std::sort(std::ranges::begin(b), std::ranges::end(b));
}
```

Lecture10-C++泛型程序设计(3) STL进阶

关联容器

关联容器分为两类:

- 有序关联容器
  - `std::set` ：有序集合
  - `std::map` ：有序映射
- 无序关联容器
  - `std::unordered_set` ：无序集合
  - `std::unordered_map` ：无序映射

有序关联容器: 元素是有序排列的，从而插入和检索只需要$O(\log n)$的时间

- `set/multiset` 在头文件 `<set>` 中
  - 集合 `set` 不允许相同元素， `multiset` 允许存在相同的元素
- `map/multimap` 在头文件 `<map>` 中
  - 映射 `map` 与 `set` 的不同在于 `map` 中存放的是成对的key/value，并根据key对元素进行排序，可快速地根据key来检索元素
  - `map` 同 `multimap` 的不同在于是否允许相同key的元素
- 无序关联容器: 元素按照特定顺序(乱序)存放，插入和检索只需要$O(1)$的时间
  - `unordered_set` 、 `unordered_multiset` 在头文件 `<unordered_set>` 中
  - `unordered_map` 、 `unordered_multimap` 在头文件 `<unordered_map>` 中

对于有序关联容器:

- 内部元素有序排列，新元素插入的位置取决于它的值，查找速度快
- 除了各容器都有的函数外，还支持以下成员函数
- `find` ：查找等于某个值的元素

    - $x < y$ 和 $y < x$ 同时不成立即为相等
- `lower_bound`：查找某个下界
- `upper_bound`：查找某个上界
- `equal_range`：同时查找上界和下界
- `count`：计算等于某个值的元素个数
    - $x < y$ 和 $y < x$ 同时不成立即为相等
- `insert`：用以插入一个元素或一个区间

`pair` 模板

```
template<typename _T1, typename _T2>
struct pair {
        typedef _T1 first_type;
        typedef _T2 second_type;
        _T1 first;
        _T2 second;
    pair(): first(), second() { }
    pair(const _T1& __a, const _T2& __b): first(__a), second(__b) { }
    template<class _U1, class _U2>
        pair(const pair<_U1, _U2>& __p): first(__p.first), second(__p.second) { }
};
```

- 第三个构造函数的用法示例

```
pair<int, int> p(pair<double, double>(5.5,4.6));
```

- `make_pair(v1, v2)`：以v1和v2值创建一个新的pair对象

```
pair<int, double> p4 = make_pair(200, 3.14);
auto p4 = make_pair(200, 3.14); // 通常可以使用auto
```

`std::multiset`

签名：

```
template<typename Key, typename Pred = less<Key>>
class multiset {
        //...
};
```

- Key表示容器中每个元素类型
- Pred类型的变量决定了multiset中的元素，“一个比另一个小”是怎么定义的
    - Pred的缺省类型是 `less<Key>`

- 使用方式
  `multiset<C> a;` 等效于 `multiset<C, less<C>> a;`

> ⚠ **Caution**
>
> 由于less模板类用<进行比较，所以这就要求类C的对象能用<比较，即重载了<运算符

`multiset` 的成员函数:

- `iterator find(const T & val);`
  - 查找等于某个值的元素，返回其迭代器; 如果找不到，返回end()
  - $x < y$ 和 $y < x$ 同时不成立即为相等
- `iterator lower_bound(const T & val);`
  - 查找一个最大的位置 `s`，使得$[begin(), it)$中所有的元素都比val小
- `iterator upper_bound(const T & val);`
  - 找一个最小的位置 `it`，使得$[it, end())$中所有的元素都比val大
- `pair<iterator,iterator> equal_range(constT&val);`
  - 同时查找上界和下界
- `int count(const T & val);`
  - 计算等于某个值的元素个数
  - $x < y$ 和 $y < x$ 同时不成立即为相等
- `iterator insert(const T & val);`
  - 将val插入到容器中并返回其迭代器
- `void insert(iterator first, iterator last);`
  - 将区间$[first, last)$插入容器

示例:

```
#include <iostream>
#include <set>
using namespace std;
template <class T>
void print(T first, T last) {
        for(; first != last; ++first) { cout << *first << " "; }
    cout << endl;
}
class A {
private:
        int n;
public:
        A(int n_): n{n_} { }
        friend bool operator<(const A& a1, const A& a2) {
        return a1.n < a2.n;
```

```
    }
        friend ostream & operator<< (ostream& os, const A& a2 ) {
                os << a2.n;
                return os;
        }
    friend class MyLess;
};
class MyLess {
public:
        bool operator()( const A& a1, const A& a2) const {// 按个位数比较大小
                return (a1.n % 10) < (a2.n % 10);
        }
};

using MSET1 = multiset<A>; // MSET1用"<"比较大小
using MSET2 = multiset<A,MyLess>; // MSET2用MyLess::operator()比较大小
// C++17要求方法是const

int main() {
    constexpr int SIZE = 6;
    A a[SIZE] = {4,22,19,8,33,40};

        MSET1 m1;
        m1.insert(a, a+SIZE);
        m1.insert(22);
        cout << "1) " << m1.count(22) << endl;
        // 输出 1) 2

        cout << "2) "; print(m1.begin(),m1.end());
        // 输出 2) 4 8 19 22 22 33 40

        MSET1::iterator pp = m1.find(19);
        if(pp != m1.end()) { // 条件为真说明找到了元素
                cout << "found" << endl;
        }
        cout << "3) ";
        cout << * m1.lower_bound(22) << "," << * m1.upper_bound(22) << endl;
        // 输出 3) 22,33

        pp = m1.erase(m1.lower_bound(22), m1.upper_bound(22));
        // pp指向被删除元素的下一个元素
        cout << "4) ";
        print(m1.begin(),m1.end());
        // 输出 4) 4 8 19 33 40

        cout << "5) ";
        cout << * pp << endl;
        // 输出 5) 33
```

```
        MSET2 m2; // m2的元素按照个位数从小到大排
        m2.insert(a, a+SIZE);
        cout << "6) ";
        print(m2.begin(),m2.end());
        //输出 6) 40 22 33 4 8 19

        return 0;
}
```

`std::set`

签名：

```
template<typename Key, typename Pred = less<Key>>
class set {
        //...
};
```

`std::set` 是无重复元素的容器

- 插入 `std::set` 中已有的元素时，忽略插入

示例：

```
#include <iostream>
#include <set>
using namespace std;
int main() {
        int a[5] = {3, 4, 6, 1, 2};
        set<int> st(a, a+5); // st中是1 2 3 4 6
        auto result = st.insert(5); // st中是1 2 3 4 5 6
        // result的类型为pair<set<int>::iterator, bool>
        // pair中的第二个元素表示是否插入成功
        if (result.second) {
                cout << *result.first << " inserted" << endl;
        } // 输出 5 inserted

        result = st.insert(5);
        if (result.second) {
                cout << *result.first << endl;
        } else {
                cout << *result.first << " already exists" << endl;
        } // 输出 5 already exists

        auto bounds = st.equal_range(4);
        // bounds 是 pair<set<int>::iterator, set<int>::iterator>
        cout << *bounds.first << "," << *bounds.second; // 输出4,5
```

```
        return 0;
}
```

`std::multimap`

签名：

```
template<typename Key, typename T, typename Pred = less<Key>>
class multimap {
        //...
        typedef pair<const Key, T> value_type;
        //...
};
```

- `multimap` 中的元素由 `<关键字, 值>` (称为“键值对”) 组成，每个元素是一个 `pair` 对象
- `multimap` 中允许多个元素的关键字相同
- 元素按照关键字升序排列，缺省情况下用 `less<Key>` 定义关键字的“小于”关系
    - 键相等的键值对的顺序是插入顺序(C++11起)
- multimap没有重载 `[]` 运算符

示例：

```
#include <iostream>
#include <map>
using namespace std;
int main() {
        multimap<int, double, less<int>> pairs;
        cout << "1) " << pairs.count(15) << endl; //求关键字等于某值的元素个数
        // 输出 1) 0

        pairs.insert({15, 2.7}); //将键值对插入到multimap中
        //此处{15, 2.7}会构造出pair<const int,double>类型的对象

        pairs.insert({15, 99.3});
        cout << "2) " << pairs.count(15) << endl;
        // 输出 2) 2

        pairs.insert({30, 111.11});
        pairs.insert({10, 22.22});
        pairs.insert({25, 33.333});
        pairs.insert({20, 9.3});
        for (auto i = pairs.begin(); i != pairs.end(); i++){
        cout << "(" << i->first << "," << i->second << ")" << ",";
        }
        // 输出 (10,22.22),(15,2.7),(15,99.3),(20,9.3),(25,33.333),(30,111.11),
```

```
    return 0;
}
```

`std::map`

签名：

```
template<typename Key, typename T, typename Pred = less<Key>>
class map {
        //...
        typedef pair<const Key, T> value_type;
        //...
};
```

- `map` 中的元素的关键字各不相同
- 可以用 `[]` 运算符通过关键字访问对应的值
    - 表达式返回的是对关键字为key的元素的值的引用
    - 如果没有关键字为key的元素，则会往容器里插入一个关键字为key的元素，其值用无参构造函数初始化，并返回其值的引用

```
map<int, double> pairs;
pairs[50] = 5;
// 会修改pairs中关键字为50的元素，使其值变成5
// 若不存在关键字等于50的元素，则插入此元素，并使其值变为5。
```

`map` 构造方法：

- 使用统一初始化构造map
- 使用基于范围的for遍历map

```
#include <iostream>
#include <string>
#include <map>
using namespace std;
int main() {
        map<string, int> scores{
        {"Alice", 90},
        {"Bob", 80}
    };
        scores["Carol"] = 75;
        // 此处auto推导出的类型为pair<const string, int>
        // 使用auto避免手写类型的错误
        for (auto& i : scores) {
        cout << i.first << " got " << i.second << " score." << endl;
```

```
    }
}
```

示例：

```
#include <iostream>
#include <map>
using namespace std;
int main() {
        map<int, double, less<int>> pairs;
        cout << "1) " << pairs.count(15) << endl;
        // 输出 1) 0

        pairs.insert({15, 2.7});
        pairs.insert({15, 99.3});
        cout << "2) " << pairs.count(15) << endl;
        // 输出 2) 1

        pairs.insert({20, 9.3});
        cout << "3) ";
        for (auto p : pairs) {
                cout << "(" << p.first << "," << p.second << ")";
        }
        cout << endl;
        // 输出 3) (15,2.7)(20,9.3)

        cout << "4) ";
        int n = pairs[40];
        for (auto p : pairs) {
                cout << "(" << p.first << "," << p.second << ")";
        }
        cout << endl;
        // 输出 4) (15,2.7)(20,9.3)(40,0)

        cout << "5) ";
        pairs[15] = 6.28;
        for (auto p : pairs) {
                cout << "(" << p.first << "," << p.second << ")";
        }
        // 输出 5) (15,6.28)(20,9.3)(40,0)

        return 0;
}
```

容器适配器

通过封装某个顺序容器，并重新组合该容器中包含的成员函数，使其满足某些特定场景的需要

- `stack`：头文件`<stack>`
  - 栈。是项的有限序列，并满足序列中被删除、检索和修改的项只能是最近插入序列的项，即按照后进先出的原则
- `queue`：头文件`<queue>`
  - 队列。插入只可以在尾部进行。删除只能在头部进行。检索和修 改只能在头或尾进行。先进先出
- `priority_queue`：头文件`<queue>`
  - 优先级队列。最高优先级元素总是第一个出列
- 容器适配器本质上还是容器，只不过此容器模板类的实现，利用了大量其它基础容器模板类中已经写好的成员函数

`std::stack`

签名：

```
template<typename T, typename Cont = deque<T> >
class stack {
        //..
};
```

stack是后进先出的数据结构，只能插入/删除/访问栈顶的元素

- 可用`vector`，`list`，`deque`来实现
  - 缺省情况下，用`deque`实现
  - 用`vector`和`deque`实现，比用`list`实现性能好
- 成员函数
  - `push`：插入元素
  - `pop`：弹出元素
  - `top`：返回栈顶元素的引用

```
stack<int> stk; //int型栈，用deque实现
//string型栈，用vector实现
stack<string, vector<string>> str_stk;
//string型栈，用vector实现，并且用向量svec初始化
stack<string, vector<string>> str_stk(svec);
```

`std::queue`

签名：

```
template< typename T, typename Cont = deque<T> > class queue {
        //...
};
```

- 和stack基本类似，可以用 `list` 和 `deque` 实现
  - 缺省情况下用 `deque` 实现
- 同样也有 `push`，`pop`，`top` 函数
  - 但是 `push` 发生在队尾
  - `pop`，`top` 发生在队头
  - 先进先出
- 有back成员函数可以返回队尾元素的引用

`std::priority_queue`

签名：

```
template < typename T,
                   typename Container = vector<T>,
                   typename Compare = less<T>
> class priority_queue;
```

- 和 `queue` 类似，可以用 `vector` 和 `deque` 实现，缺省情况下用 `vector` 实现
- `priority_queue` 通常用堆排序技术实现，保证最大的元素总是在最前面
  - `pop` 删除最大的元素
  - `top` 返回最大元素的引用
  - 默认的元素比较器是 `less<T>`

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
        priority_queue<double> pq1;
        pq1.push(3.2);
        pq1.push(9.8);
        pq1.push(9.8);
        pq1.push(5.4);
        while (!pq1.empty()) {
                cout << pq1.top() << " ";
                pq1.pop();
        }
        cout << endl;
        //上面输出 9.8 9.8 5.4 3.2

        priority_queue<double,vector<double>,greater<double>> pq2;
        pq2.push(3.2);
        pq2.push(9.8);
        pq2.push(9.8);
        pq2.push(5.4);
        while (!pq2.empty()) {
```

```
                cout << pq2.top() << " ";
                pq2.pop();
        }
        //上面输出 3.2 5.4 9.8 9.8

        return 0;
}
```

输入输出流

C++中与输入输出相关的类

- `istream` 是用于输入的流类，cin 就是该类的对象
- `ostream` 是用于输出的流类，cout 就是该类的对象
- `ifstream` 是用于从文件读取数据的类
- `ofstream` 是用于向文件写入数据的类
- `iostream` 是既能用于输入，又能用于输出的类
- `fstream` 是既能从文件读取数据，又能向文件写入数据的类

标准流对象

- 输入流对象
  - `cin` 与标准输入设备相连，对应于标准输入流，用于从键盘读取数据，也可以被重定向为从文件中读取数据
- 输出流对象
  - `cout` 与标准输出设备相连，对应于标准输出流，用于向屏幕输出数据，也可以被重定向为向文件写入数据
  - `cerr` 与标准错误输出设备相连
  - `clog` 与标准错误输出设备相连
  - `cerr` 和 `clog` 的区别在于 `cerr` 不使用缓冲区，直接向显示器输出信息; 而输出到 `clog` 中的信息先会被存放在缓冲区，缓冲区满或者刷新 时才输出到屏幕

如何判断输入流结束?

可以用如下方法判输入流结束

- 如果从键盘输入, 则输入Ctrl+Z代表输入流结束

```
int x;
while (cin >> x) {
        //...
}
```

- 原理

    - istream重载>>运算符的定义

```
istream& operator>>(int &a) { //...
        return *this;
}
```

- 在istream或其基类里重载了 `explicit operator bool`
    - 为什么使用explicit?

istream类的成员函数:

- `istream& getline(char * buf, int bufSize);`
    - 从输入流中读取$bufSize-1$个字符到缓冲区 `buf`，或读到 `'\n'` 为止
- `istream& getline(char * buf, intbufSize, char delim);`
    - 从输入流中读取$bufSize-1$个字符到缓冲区 `buf`，或读到 `delim` 为止
        - 以上两个函数都会自动在 `buf` 中读入数据的结尾添加 `'\0'`
        - `'\n'` 或 `delim` 都不会被读入 `buf`，但会被从输入流中取走
- `bool eof();`：判断输入流是否结束
- `int peek();`：返回下一个字符，但不从流中去掉
- `istream& putback(char c);`：将字符ch放回输入流

流操纵算子

使用流操纵算子需要 `#include <iomanip>`

- 整数流的基数
    - 流操纵算子 `dec`，`oct`，`hex`，`setbase`
- 浮点数的精度
    - 成员函数 `precision`
    - 流操纵算子 `setprecision`
    - 流格式操纵算子 `setiosflags`
- 设置域宽
    - 成员函数 `setw`
    - 流操纵算子 `width`

我们也可以自定义流操纵算子

```
ostream &tab(ostream &output) {
        return output << '\t';
}
cout << "aa" << tab << "bb" << endl;
```

原理

- `iostream` 里对 `<<` 进行了重载(成员函数)

```
ostream &operator<<(ostream &(*p)(ostream &));
```

- `hex`，`dec`，`oct` 等流操纵算子都是函数

输入输出迭代器

类模板 `std:: istream_iterator` 用于定义输入迭代器对象

- 使用运算符 `>>` 从输入流读入连续的元素
- 支持比较(`==`，`!=`)，解引用(`*`，`->`)，自增(`++`)

示例：

```
#include <iostream>
#include <algorithm>
#include <iterator>// istream_iterator定义于此
using namespace std;
int main() {
        istream_iterator<int> inputInt(cin);
        // inputInt构造时从输入流中读取一个数据保存到其中
        int n1, n2;
        n1 = *inputInt; // 从迭代器中获取读到的数据赋值给n1
        inputInt++; // ++是从输入流中读取一个数据保存到其中
        n2 = *inputInt; // 从迭代器中获取读到的数据赋值给n2
        cout << n1 << ", " << n2 << endl;

        return 0;
}
```

类模板 `std::ostream_iterator` 用于定义输出迭代器对象

- 该对象使用运算符 `<<` 将连续的元素写入输出流
- 支持解引用(`*`，`->`)，自增(`++`)

示例：

```
#include <iostream>
#include <iterator>
using namespace std;
int main() {
        ostream_iterator<int> iter(cout);
        *iter = 1;
        ++iter;
        *iter = 2;
        ++iter;
```

```
        *iter = 3;
        ++iter;
        // 输出 123
}
```

```
#include <iostream>
#include <iterator>
#include <vector>
using namespace std;
int main() {
    ostream_iterator<int> iter(cout);
    vector v{1, 2, 3};
    for (auto i = v.begin(); i != v.end(); ++i) {
                *iter = *i;
        }
        // 输出 123
}
```

`std::copy` 可以用于从输入流读取数据到容器或向输出流写入容器中的数据

定义：

```
template <class InputIt, class OutputIt>
OutputIt copy(InputIt first, InputIt last, OutputIt d_first) {
        for (; first != last; ++first, ++d_first) {
                *d_first = *first;
        }
    return d_first;
}
```

示例：

```
#include <iostream>
#include <iterator>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
istream_iterator<int> begin_iter(cin);
istream_iterator<int> end_iter;
vector<int> v{};
copy(begin_iter, end_iter, back_inserter(v));
    ostream_iterator<int> iter(cout);
    copy(v.begin(), v.end(), iter);
}
```

注意：

- `std::ostream_iterator` 还可提供额外的第二实参，表示输出元素后的分隔字符串

```
vector a{1, 3, 5, 7, 9};
// 输出 a 中的元素，以*分隔
ostream_iterator<int> outputInt(cout, "*");
copy(a.begin(), a.end(), outputInt);
```

std::string

- `string` 类是一个模板类，它的定义如下 `using string = basic_string<char>;`
- 使用 `string` 类要包含头文件 `<string>`
- `string` 对象的常用初始化:
  - `string s1("Hello");`
  - `string month = "March";`
  - `string s2(8,'x’);`
- `string` 类不提供以字符和整数为参数的构造函数
  - 错误的初始化方法:
    - `string error1 = 'c’;`
    - `string error2('u');`
    - `string error3(8);`
- 可以将字符赋值给string对象
  - `s = 'n';`

注意:

- 类型推断:
  - `auto str = "hello";` 推断为 `const char*`
- 使用 `""s` 表示 `std::string` 类型字面量
  - 需要 `using namespace std::literals` 才可使用
  - `auto str = "hello"s;` 推断为 `string`
- `string` 类的输入输出
  - 支持流插入运算符 `cin >> s;`
  - 支持流提取运算符 `cout << s;`
  - 支持 `getline` 函数 `getline(cin, s);`
- `string` 类的容量
  - `string` 对象的长度使用成员函数 `length()` 读取
    - 也可以用成员函数 `size()` 读取
  - 检查 `string` 是否无字符使用 `empty()` 成员函数
  - `capacity()` 成员函数是返回当前对象分配的存储空间能保存的字符数量
- 访问 `string` 对象中的某个字符
  - `operator[]` 函数: `s[i]`

    - 成员函数 `at()`：`s.at(j)`
        - 成员函数 `at()` 会做范围检查，如果超出范围，会抛出 `out_of_range` 异常，而下标运算符 `[]` 不做范围检查
- 赋值
    - 用 `=` 赋值
    - 用成员函数 `assign()` 赋值
- 连接
    - 用 `+` / `+=` 运算符连接字符串
    - 用成员函数 `append` 连接字符串
- 子串
    - 成员函数 `substr`
- 比较
    - 用关系运算符比较 `string` 的大小: `==`，`>`，`>=`，`<`，`<=`，`!=`
    - 用成员函数 `compare()` 比较 `string` 的大小
    - 比较规则
- 查找
    - 成员函数 `find()`
    - 成员函数 `rfind()`
    - 成员函数 `find_first_of()`
    - 成员函数 `find_last_of()`
- 删除:成员函数 `erase()`
- 替换:成员函数 `replace()`
- 插入:成员函数 `insert()`

字符串流处理

除了标准流和文件流输入输出外，还可以从 `string` 进行输入输出

类似 `istream` 和 `osteram` 进行标准流输入输出，用 `istringstream` 和 `ostringstream` 进行字符串上的输入输出，也称为内存输入输出

- `istringstream` 和 `ostringstream` 在 `<sstream>` 头文件中

字符串输入流

```
#include <string>
#include <iostream>
#include <sstream>
using namespace std;
int main() {
        string input("Input test 123 4.7 A");
        istringstream inputString(input);
        string string1, string2;
```

```
        int i;
        double d;
        char c;
        inputString >> string1 >> string2 >> i >> d >> c;
        cout << string1 << endl << string2 << endl;
        cout << i << endl << d << endl << c <<endl;
        long L;
        if(inputString >> L) {
                cout << "long\n";
        } else {
                cout << "empty\n";
        }
        return 0;
}
```

字符串输出流

```
#include <string>
#include <iostream>
#include <sstream>
using namespace std;
int main() {
        ostringstream outputString;
        int a = 10;
        outputString << "This " << a << " ok" << endl;
        string s = outputString.str();
        cout << s;
        return 0;
}
```

Lecture11-C++多文件工程

回顾：程序设计语言

- 机器语言: 由二进制的机器指令构成，可直接执行。
- 汇编语言: 为便于记忆，将机器指令用助记符和符号写成的程序
- 高级语言: 用易于理解的形式，如英文单词、数学公式，按严格的语法规则和一定的逻辑关系写出的程序集合

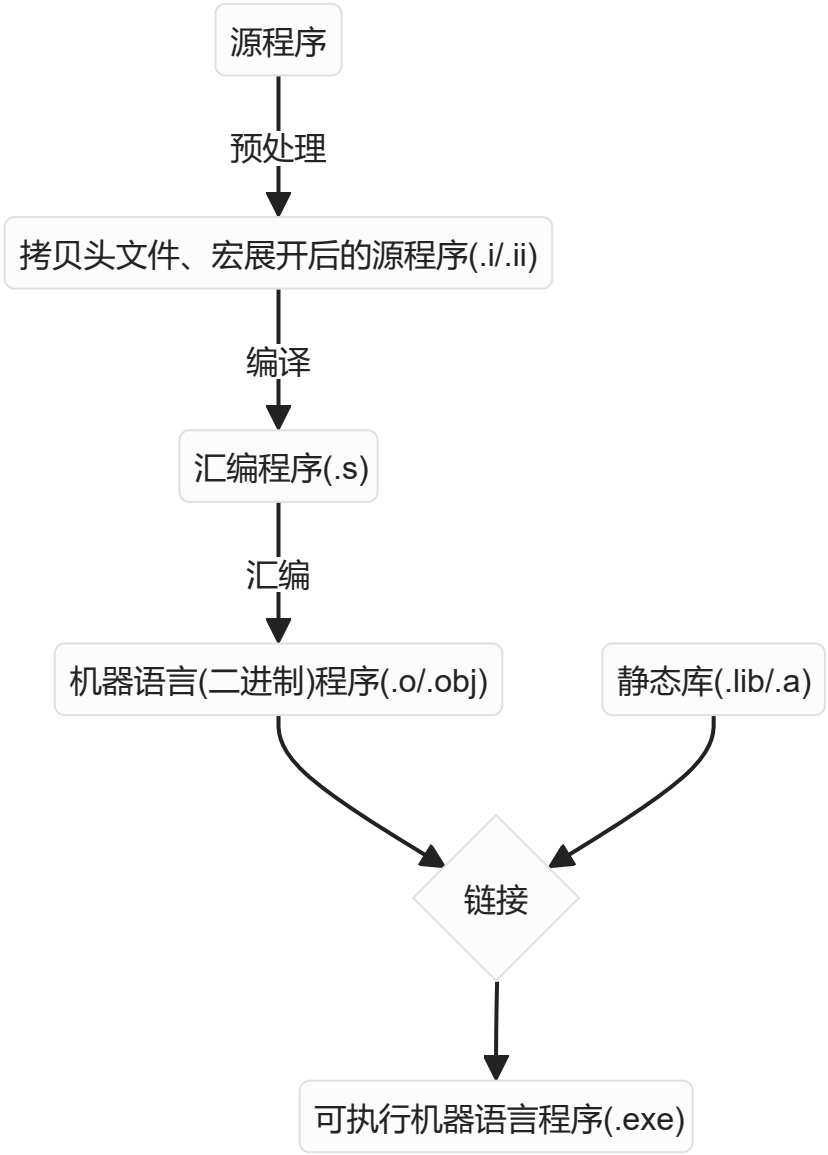
那么，如何把高级语言变为机器语言呢?

- 语言处理程序
  - 源程序: 用汇编语言或高级语言各自规定的符号和语法规则编写的程序
- 目标程序: 将计算机源程序翻译成机器语言后的程序
- 源程序翻译成目标程序有两种: 编译方式和解释方式
  - 编译方式: C++, ...

- 解释方式: Python, ...
- 优缺点?
  - 编译方式: 编译时间长，执行速度快，跨平台性差
  - 解释方式: 编译时间短，执行速度慢，跨平台性好

C++的“翻译”过程

源程序（.cpp/.c/.h）



那么，当我们写了很多代码时候，用什么方法可以只修改一个函数的实现而无需重新编译其他函数?

- 拆分代码文件：拆分为多个代码文件后，当修改某个代码文件时，只需重新编译改过的文件并重新链接即可，整体的翻译效率得到提升

C++多代码文件的组织

C++项目的代码文件分类

一个完整的c++项目常常是由多个代码文件组成，根据后缀名不同，大致分为两类

- .h 头文件(Header file)一般只存放类型的定义和符号的声明(在不涉及模板的情形下)
  - 由于头文件只含声明，所以单独编译头文件不会产生任何二进制指令(只有定义才可以让编译产生结果)
  - 头文件的作用仅在于被 `.cpp` 文件所包含
- .cpp 实现文件(Implementation file)存放对应头文件声明的定义
  - 为了引入相关类型的定义，一般需要将同名头文件也包含进来
  - 每一个实现文件一般都对应着一个翻译单元，这些翻译单元会在最后链接为整个执行程序
  - C++项目一般会包含一个带有 `main()` 函数的特殊实现文件，作为程序的执行入口，该实现文件一般只会包含必需的头文件

示例：

student.h

```
#ifndef STUDENT_H
#define STUDENT_H
#include <string>
class Student {
public:
        Student(std::string name, int age);
        void display();
private:
        std::string name;
        int age;
};
#endif
```

student.cpp

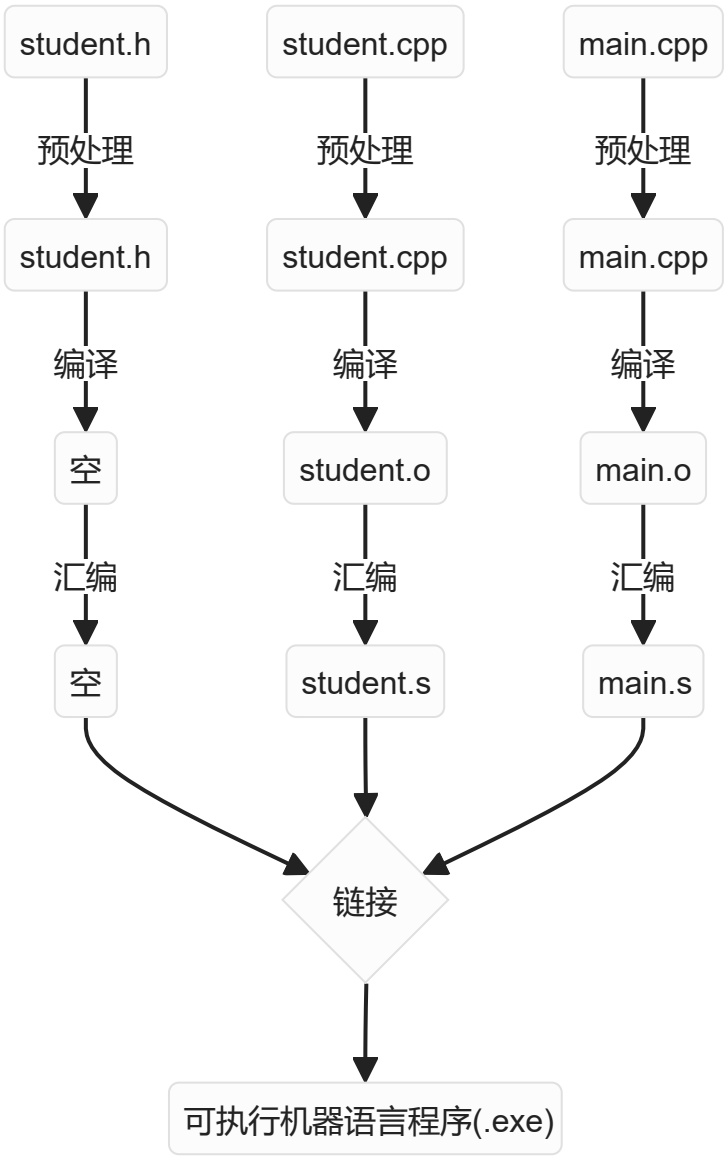
```
#include "student.h"
#include <iostream>
Student::Student(std::string name, int age): name(name), age(age) { }
void Student::display() {
        std::cout << name << " " << age << std::endl;
}
```

main.cpp

```
#include "student.h"
int main() {
        Student s("Tom", 20);
        s.display();
```

```
        return 0;
}
```

编译时，原理如下：



预处理指令

cpp文件中以`#`开头且以换行符为结尾的语句是预处理指令，在编译之前完成特定的替换任务

- `#include`预处理指令
  - 把某个源文件直接插入进指令所在的位置
    - 效果类似于文件内容拷贝
  - 预处理时不断递归地展开`#include`指令，直至不存在`#include`指令
- `#include <头文件名>`
  - 优先在系统库文件中查找
  - `<iostream>``<cmath>`等库就是位于系统库路径的名为`iostream`、`cmath`等的文件
- `#include "文件名"`
  - 优先在当前源文件所在的路径中查找

Question

为什么上面代码中我使用了 `ifndef` 与 `define`？

- 防止头文件被重复包含
  - 如果一个头文件被重复包含，编译器会报错
- 一般头文件的内容是类型的声明和符号的声明，不会有定义
  - 如果一个头文件被重复包含，会导致符号重复定义

为了避免重复包含，我们可以使用利用预处理指令

- `#define`
- `#if`
- `#elif`
- `#else`
- `#endif`
- `#ifdef`
- `#ifndef`
  使编译器选择性地编译某一部分而不编译其它部分

Tip

头文件中的 `#ifndef` 与 `#define` 的作用是为了防止头文件被重复包含，这是一种常见的防范措施，称为“头文件保护”。
事实上，我们还可以使用 `#pragma once` 来代替 `#ifndef` 与 `#define`， `#pragma once` 是一种更简洁的写法，但是 `#pragma once` 是非标准的写法，不是所有编译器都支持。

条件编译

若常量表达式为非0值，则编译代码1，但不编译代码2， `#else` 可以省略

- 如果连续多个 `#else`，`#if`，可以使用 `#elif` 来简化

```
#if 常量表达式
// 代码1
#else
// 代码2
#endif
```

示例：判断编译器版本

```
#if __cplusplus >= 201103L
int* p = nullptr;
#else
int* p = NULL;
#endif
```

`#defined` 运算符可以返回当前环境是否定义了某个宏

- 若宏存在，则值为1; 若宏不存在，则值为0

```
#if defined ONLINE_JUDGE
cout <<"welcome";
#endif
```

- `#ifdef` 等价于 `#if defined`， `#ifndef` 等价于 `#if !defined`

```
#ifdef ONLINE_JUDGE
// ...
#ifndef ONLINE_JUDGE
// ...
```

对于项目：

student.h

```
#ifndef STUDENT_H
#define STUDENT_H
class Student;
#endif
```

school.h

```
#ifndef SCHOOL_H
#define SCHOOL_H
#include "student.h"
class School;
#endif
```

main.cpp

```
#include "student.h"
#include "school.h"
// ...
```

现在，我们就不用担心重复包含头文件的问题了，虽然我们的项目`main.cpp`文件中可能会`#include`了2次`"student.h"`，但鉴于`STUDENT_H`宏只能定义一次，所以`Student`类也仅会定义一次

命名空间

我们还需要考虑这样一个场景，如果我们的项目中有两个类，两个类中都有一个同名的常量，那么我们该如何解决这个问题呢？

例如：

student.h

```
#ifndef STUDENT_H
#define STUDENT_H
const int MAX {100};
#endif
```

school.h

```
#ifndef SCHOOL_H
#define SCHOOL_H
const int MAX {200};
#endif
```

main.cpp

```
#include "student.h"
#include "school.h"
int main() {
        std::cout << MAX << std::endl; // error, program will not compile
}
```

我们如何解决？

- 使用命名空间
  - 命名空间是C++中的一种用于避免命名冲突的机制
  - 命名空间可以将一组标识符封装在一个作用域中，不同命名空间中的标识符不会发生冲突
  - 命名空间的定义可以分开，可以在不同的文件中定义
  - 命名空间可以嵌套定义

student.h

```
#ifndef STUDENT_H
#define STUDENT_H
namespace StudentNS {
        const int MAX {100};
}
#endif
```

school.h

```
#ifndef SCHOOL_H
#define SCHOOL_H
namespace SchoolNS {
        const int MAX {200};
}
#endif
```

main.cpp

```
#include "student.h"
#include "school.h"
int main() {
        std::cout << StudentNS::MAX << std::endl;       // 100
        std::cout << SchoolNS::MAX << std::endl;        // 200
        std::cout << MAX << std::endl;                          // error
}
```

- 自定义命名空间
  - 命名空间一般在.h头文件中
  - 当类的声明位于指定的命名空间中时，如果要在类的外部实现其成员函数，需同时注明所在命名空间名和类名
  - 不同的头文件中可以使用不同的命名空间，也可以使用相同的命名空间，但后者的前提是位于该命名空间中的成员必须保证互不相同
- 作用域解析运算符 `::`
  - 基本用法: `命名空间名::名字`
  - `using namespace 命名空间名;`
  - `using 命名空间名::名字;`

单一定义原则

声明与定义

- 声明:仅仅告诉编译器该符号存在，至于该符号具体含义，等链接的时候才能知道
- 定义:某个符号完整的描述清楚，是变量还是函数，变量类型以及变量值是多少，函数的返回值是什么等

单一定义原则(One Definition Rule, ODR)是一系列规则，规定了C++如何处理声明和定义，比较重要的包括:

- 一个翻译单元中，允许出现变量、函数、类型和模板的多次声明，但最多只允许出现一次定义
- 一个翻译单元中，如果ODR-使用了(即需要取其地址的行为)一个符号，那么它最少要出现一次定义
- 整个程序中，非内联的符号最多只允许出现一次定义

多个文件涉及同一变量名的处理

- 多个文件共享全局非只读变量
  - 在一个文件中定义，其他文件中用`extern`声明
- 多个文件共享全局只读变量
  - 在一个文件中用`extern const`定义，其他文件中用`extern const`声明
- 多个文件各自使用全局只读变量
  - `const`默认表示变量被各文件分别管理

命令行

- 当前操作系统大多默认以图形界面(Graphical User Interface，GUI)作为人机交互的方式
- 早期的计算机，使用者只能在一个由纯文本组成的界面中，使用键盘向其中键入“命令”，然后得到计算机显示的结果，称为命令行界面(Command- Line Interface，CLI)
- 使用终端模拟器(Terminal Emulator)在GUI上使用CLI
- 终端模拟器启动时会打开一个称为Shell的程序，该程序会不断地接受键盘的输入(被称作“命令”)，然后分析这个输入是在做什么，然后做一些事情
  - Windows:cmd
  - GNU/Linux:bash
  - macOS:zsh

命令行参数

- 命令的格式
  - `程序名[空格]参数1 [空格]参数2[空格]...·、`
  - 每一个命令按空格分成若干段
    - 第一段称作“程序名”，其指定了一个程序，Shell会在CLI中启动这个程序，然后将输入输出的控制权交给它
    - 剩下的部分称为命令的“参数”，这些参数将在程序启动时通过一种特别的方式传入程序，而程序可以通过一些方法获得这些参数
- 工作路径和PATH
  - Shell需要在文件系统的一个特定的路径中工作，称为工作路径，一般作为输入提示符来呈现
    - 可以通过`cd 路径名`来更换工作路径

- `.` 代表当前路径; `..` 代表上层路径

```
chenxub@chenxu-PC:/home/chenxub/$ cd ./project/pku-software
```

- 操作系统提供了一些常用的命令行程序，为了调用它们的时候不写路径名，因此操作系统为这些路径提供了快捷方式——PATH环境变量
  - PATH 的程序在执行时只需给出程序名，而不用给出其路径

之后，我们通过CLI启动程序时，可以在程序名后面加上一些参数，这些参数会被传递给程序，程序可以通过一些方法获得这些参数

C/C++程序利用 `main` 函数的参数获取命令行参数

- argc: 代表启动程序时，命令行参数的个数。C/C++ 语言规定，可执行程序本身的文件名，也算一个命令行参数，因此argc 的值至少是1
- argv: 指针数组，其中的每个元素都是一个char*类型的指针，该指针指向一个字符串，这个字符串里就存放着命令行参数 (argv[0]始终是程序名本身)

```
#include <iostream>

int main(int argc, char** argv) {
        if (argc == 1) {
                std::cout << "No command line argument\n";
        } else {
                std::cout << "Command line arguments are:\n";
                for (int i = 1; i < argc; i++) {
                        std::cout << argv[i] << std::endl;
                }
        }
}
```

- 命令行重定向
  - 重定向输出(`>`)
  - 将标准输出重定向到 `out.txt` 文件中
- 重定向输入(`<`)
  - 将 `in.txt` 文件中的内容重定向到标准输入中
- Windows的PowerShell不支持这种形式的重定向

命令行编译

以g++为例

目录内文件层级如下

```
── project
        ├─ main.cpp
        ├─ student.cpp
        ├─ student.h
        ├─ school.cpp
        └─ school.h
```

```
g++ main.cpp student.cpp school.cpp -o main
```

- 当指定多个翻译单元时，编译器会分别编译这些翻译单元，并在编译完成后自动调用链接器来链接它们，最终将链接的结果转换为可执行文件保存到 `./main.exe` 中
- `-o` 选项指定了编译的结果存放在哪里
- `g++` 通过文件后缀名来判断其流程的起点

后缀名	g++ 的行为
.cpp .c++ .cc .cxx 等	视为源文件，将预处理作为第一步
.ii	视为预处理后的源文件，将编译作为第一步
.s .S 等	视为汇编文件，将汇编作为第一步
其它(一般为 .o .obj)	视为对象文件，将链接作为第一步

选项	g++ 的行为
-E	将预处理作为最后一步，得到预处理后的源文件
-S	将编译作为最后一步，得到汇编文件
-c	将汇编作为最后一步，得到对象文件
不含以上选项	将链接作为最后一步，得到可执行文件

> ⚠ **Caution**
>
> `-E` 并不通过 `-o` 选项来控制输出位置，它直接将预处理结果输出到屏幕(stdout)上。所以可使用重定向运算符 `>` 将其输出保存到想要的位置 `./hello.ii` 中

```
g++ ./hello.cpp -E > ./hello.ii          # 预处理
g++ ./hello.ii -S -o ./hello.s           # 编译
g++ ./hello.s -c -o ./hello.o            # 汇编
g++ ./hello.o -o ./hello                 # 链接
./hello                                  # 执行
```

GNU Make与Cmake

GNU Make

GNU Make(简称Make)是一个经典的构建工具

- “构建”(Build)是指在一个项目中安排编译的顺序、时机，最终得到想要的结果(称为“目标”(Target))
- Make使用一种称为Makefile的文件来指明这些构建方法
  - Makefile就是文件名为 `makefile` 的文件，不带后缀名
- Makefile的格式
  - 生成命令前面的空白是一个 `Tab` 字符，而不是若干个空格

格式：

```
目标: 依赖
        命令
```

例如，对于下面的代码库

```
── project
        ├─ main.cpp
        ├─ student.cpp
        ├─ student.h
        ├─ school.cpp
        └─ school.h
```

我们可以写出如下的makefile

```
main: main.o student.o school.o
        g++ main.o student.o school.o -o main
main.o: main.cpp student.h school.h
        g++ -c main.cpp -o main.o
student.o: student.cpp student.h
        g++ -c student.cpp -o student.o
school.o: school.cpp school.h
        g++ -c school.cpp -o school.o
clean:
        rm -f main main.o student.o school.o
```

在CLI中执行 `make 目标文件` 完成构建

- Make可以通过每个文件的最后改动时间来确定一个目标是否需要重新生成，而不用再手动判断
- 通过 Make 来管理构建方法是一种非常原始的策略
  - 翻译单元越多，编写 `Makefile` 的难度就越大

CMake

- CMake可以通过一个名为 `CMakeLists.txt` 的脚本来生成 Makefile
- 第一行规定了最低CMake版本
- 第二行指定了项目名称
- 第三行 `add_executable` 告诉 CMake，最终要生成一个可执行文件，可执行文件的名字叫 `main`，它需要从 `main.cpp` `f.cpp` `g.cpp` 三个文件编译链接得到

```
cmake_minimum_required(VERSION 3.18.0) project(HelloWorld)
add_executable(main main.cpp f.cpp g.cpp)
CMakeLists.txt
```

- 运行 `cmake` 命令可根据 `CMakeLists.txt` 生成一系列文件，其中包含 `Makefile`
- 再运行 `make` 命令来生成可执行文件

`CMakeLists.txt` 中的常用命令

- `add_executable(tgt srcs...)`
  - 从指定的源文件编译出一个可执行文件
- `target_include_directories(tgt PRIVATE dirs...)`
  - 添加包含目录
- `target_link_directories(tgt PRIVATE dirs...)`
  - 添加库查找目录
- `target_link_libraries(tgt PRIVATE libs...)`
  - 添加库链接(-l...)
- `target_compile_options(tgt PRIVATE opts...)`
  - 设置编译选项
- `set_target_properties(tgt PROPERTIES prop1 val1 prop2 val2...)`
  - 设置其它的编译属性(标准版本、输出路径等等)

文件读写

- 操作系统以文件为单位对外存进行管理
  - 当访问磁盘等外存上的数据时，必须先按文件名找到指定的文件，然后再从该文件中读取数据
  - 如果要在外存上存储数据，也必须先建立一个文件，才能向其存入数据
- 文件通过目录来组织和管理
  - 目录一般采用树状结构，在这种结构中，每个磁盘有一个根目录，它包含若干文件和子目录
  - 目录中可以包含文件或目录
- 路径：用于定位一个文件或者目录的字符串
  - 绝对路径：完整的描述文件位置的路径，包含绝对路径的文件名由磁盘驱动器、目录层次和文件名三部分组成
  - 相对路径：目标文件相对于源文件所在目录的路径

- 当前目录：省略或 `.\`
- 下一级目录：下一级目录名`\`或`.\`下一级目录名`\`
- 上一级目录：`..\`

```
D:\course-23spring\pku-software\01.py
/usr/example/01.py
```

文件1的绝对路径	文件2的绝对路径	文件2相对于文件1的相对路径
/pku/cpp/01.cpp	/pku/cpp/d.txt	d.txt or ./d.txt
	/pku/cpp/data/d.txt	data/d.txt or ./data/d.txt
	/pku/d.txt	../d.txt
	/d.txt	../../d.txt
	/data/d.txt	../../data/d.txt

文件和流

- 将所有记录顺序地写入一个文件，称为顺序文件
- 可以将顺序文件看作一个有限字符构成的顺序字符流，然后像对`cin`，`cout`一样的读写

用法

```
#include <fstream> // 包含头文件
ofstream outFile("clients.dat", ios::out | ios::binary); //打开文件
```

- 创建文件
  - `ofstream`是`fstream`中定义的类
  - `outFile`是程序定义的`ofstream`类的对象
  - `"clients.dat"` 是将要建立的文件的路径
  - `ios::out` 是打开并建立文件的选项
    - `ios::out` 输出到文件，删除原有内容
    - `ios::app` 输出到文件，保留原有内容，总是在尾部添加
    - `ios::ate` 输出到文件，保留原有内容，在文件任意位置添加
  - `ios::binary` 以二进制文件格式打开文件
- 也可以先创建`ofstream`对象，再用`open`函数打开

```
ofstream fout;
fout.open("test.out", ios::out | ios::binary);
```

文件的读写指针

标识文件操作的当前位置，该指针在哪里，读写操作就在哪里进行

```
ofstream fout("a1.out", ios::ate);
longlocation = fout.tellp(); //取得写指针的位置
location = 10L;
fout.seekp(location); //将写指针移动到第10个字节处
fout.seekp(location, ios::beg);//从头数location
fout.seekp(location, ios::cur);//从当前位置数location
fout.seekp(location, ios::end);//从尾部数location
// location 可以为负值
// istream 类对象的读指针函数 seekg() 用法也类似
```

文件的关闭

- 流对象提供close成员函数用于关闭文件
- 几乎任何情形都不需要手动调用流对象的close成员函数
- 尽管部分教材可能会要求这样做，但这是完全没有必要的，因为文件的关闭操作会在 `std::*fstream` 的析构函数中自动完成，无需任何手动处理

字符文件的读写

- 因为文件流也是流，所以前面讲过的流的成员函数和流操作算子也同样适用于文件流

例：写一个程序，将文件 `in.txt` 里面的整数排序后，输出到 `out.txt`

```
#include<iostream>
#include<fstream>
#include<vector>
#include<algorithm>
using namespace std;

int main() {
        vector<int> v;
        ifstream srcFile("in.txt", ios::in);
        ofstream destFile("out.txt", ios::out);
        int x;
        while(srcFile>>x){
                v.push_back(x);
        }
        sort(v.begin(), v.end());
        for (int i= 0; i < v.size(); i++){
                destFile << v[i] << " ";
        }
        return 0;
}
```

二进制文件读写

二进制读文件

```
istream &read(char *s, long n);
```

- 将文件读指针指向的地方的 `n` 个字节内容，读入到内存地址 `s`，然后将文件读指针向后移动 `n` 字节
  - 以 `ios::in` 方式打开文件时，文件读指针开始指向文件开头

二进制写文件

```
ostream &write(const char *s, long n);
```

- 将内存地址 `s` 处的 `n` 个字节内容，写入到文件中写指针指向的位置，然后将文件写指针向后移动 `n` 字节
  - 以 `ios::out` 方式打开文件时，文件写指针开始指向文件开头
  - 以 `ios::app` 方式打开文件时，文件写指针开始指向文件尾部

例：从键盘输入几个学生的姓名的成绩，并以二进制文件形式保存

```
struct Student {
        char name[20];
        int score;
};

int main() {
        Student stu;
        ofstream outFile("students.dat", ios::out | ios::binary);
        while (cin >> stu.name >> stu.score) {
                outFile.write((char*)&stu, sizeof(Student));
        }
        return 0;
}
```

输入：

```
Tom 60
Jack 80
Jane 40
```

形成的 `students.dat` 文件大小为72字节

例：将 `students.dat` 文件的Jane的名字改成Mike，并在屏幕上输出文件的内容

```
struct Student {
        char name[20];
        int score;
```

```
};

int main() {
        Student stu;
        fstream ioFile("students.dat", ios::in | ios::out | ios::binary);
        if (!ioFile) {
                cout << "error";
                return 0;
        }

        iofile.seekp(2 * sizeof(stu), ios::beg); // 定位写指针到第3个记录
        iofile.write("Mike", strlen("Mike"));
        iofile.seekg(0, ios::beg); // 定位读指针到开头
        while (iofile.read((char *)&stu, sizeof(stu))){
                cout << stu.name << " " << stu.score << endl;
        }
        return 0;
}
```